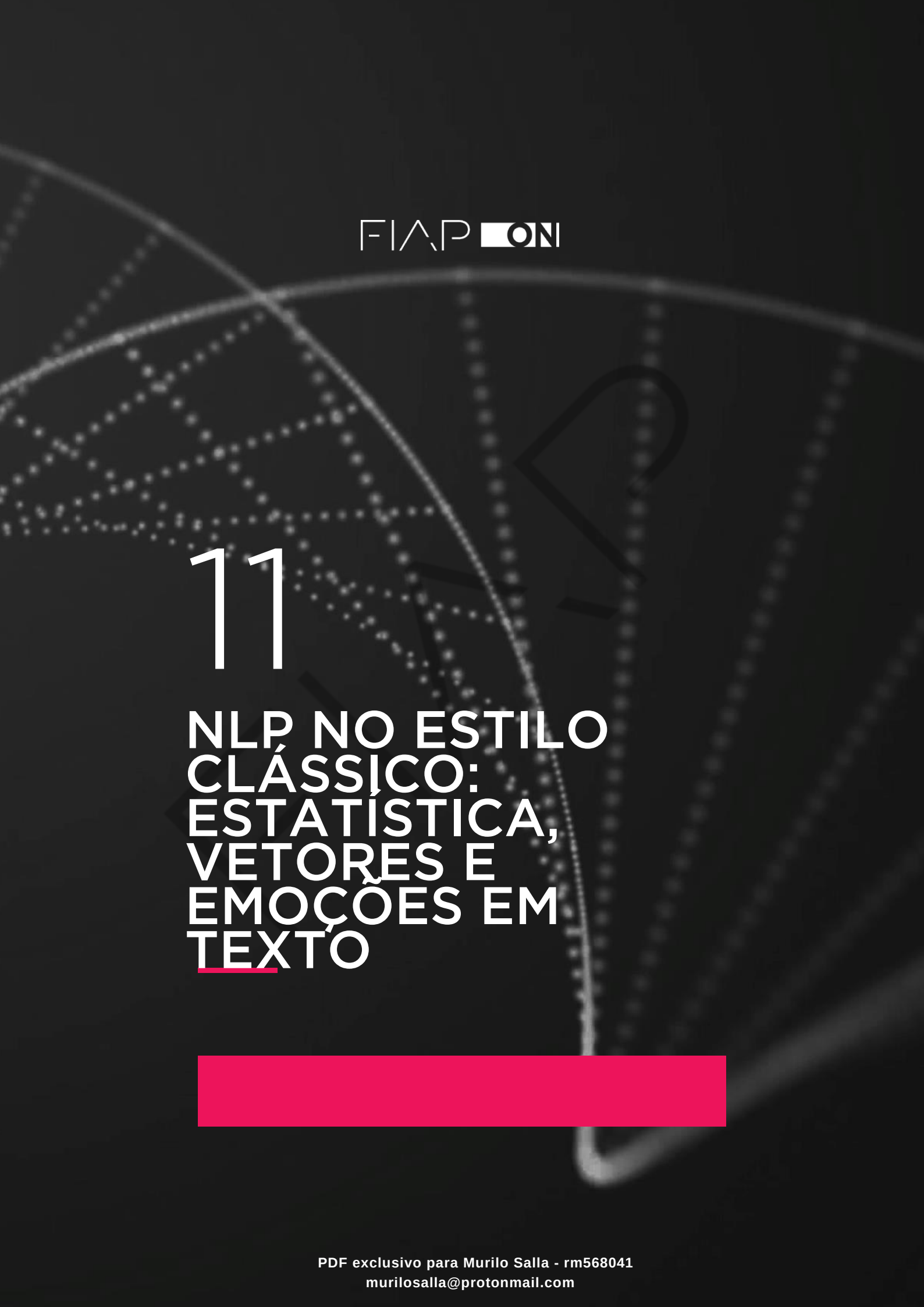


11

NLP NO ESTILO CLÁSSICO: ESTATÍSTICA, VETORES E EMOÇÕES EM TEXTO

PDF exclusivo para Murilo Salla - rm568041
murilosalla@protonmail.com

LISTA DE FIGURAS

Figura 1 - Saída do código-fonte 2, histograma e distribuição 12
Figura 2 - Saída do código-fonte 3, distribuição do lançamento de uma moeda e de vários lançamentos de moedas.....16
Figura 3 - Exemplo de uma Cadeia de Markov 24
Figura 4 - Exemplo de uma Cadeia de Markov Oculta.....25
Figura 5 - Resultado do código-fonte 9, uso do NER-HMM 28
Figura 6 - Resultado do código-fonte 10, uso do LDA.....31
Figura 7 - Representação gráfica da execução do OHE 32
Figura 8 - Resultado do código-fonte 11, uso do OHE.....33
Figura 9 - Resultado do código-fonte 12, uso do BoW.....35
Figura 10 - Resultado do código-fonte 13, uso do TF-IDF 37
Figura 11 - Esquemas de treinamento do Word2Vec.....38
Figura 12 - Resultado do código-fonte 14, uso do Word2Vec.....40
Figura 13 - Resultado do código-fonte 16, operação vetorial com Word2Vec.....42
Figura 14 - Resultado do código-fonte 17, carregando dataset SPAM 46
Figura 15 - Resultado do código-fonte 18, dataset SPAM vetorizado.....47
Figura 16 - Resultado do código-fonte 19, relatório de desempenho da classificação 48
Figura 17 - Resultado do código-fonte 20, análise de sentimento com NLTK VADER 49
Figura 18 - Resultado do código-fonte 21, acurácia de classificação em função do tamanho do vocabulário do BoW 53
Figura 19 - Resultado do código-fonte 22, representação da redução de dimensionalidade por Truncated SVD (LSA), seguido de T-SNE.....55
Figura 20 - Resultado do código-fonte 23, representação dos grupos encontrados pelo DBSCAN sobre o espaço projetado do T-SNE.....57
Figura 21 - Resultado do código-fonte 24, apresentado as 10 primeiras frases do cluster 17.....57
Figura 22 - Resultado do código-fonte 24, apresentado as 10 primeiras frases do cluster 9.....58

TEXTO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Aprendizado de máquina clássico com Scikit-Learn em Python.....10
Código-fonte 2 - Análise exploratória e estatística descritiva em Python 12
Código-fonte 3 - Exemplo de distribuições estatísticas 15
Código-fonte 4 - Modelo n-gram implementado em Python 21
Código-fonte 5 - Uso do n-gram para prever probabilidade de uma sentença.....22
Código-fonte 6 - Uso do n-gram para prever a próxima palavra 22
Código-fonte 7 - Uso do n-gram para prever a próxima palavra usando corpus.....23
Código-fonte 8 - Implementação de módulo NER através de HMM em Python.....27
Código-fonte 9 - Demonstração do NER-HMM em Python 28
Código-fonte 10 - Demonstração do LDA usando Gensim 31
Código-fonte 11 - Demonstração do OHE usando Sklearn.....33
Código-fonte 12 - Demonstração do BoW usando Sklearn.....34
Código-fonte 13 - Demonstração do TF-IDF usando Sklearn 37
Código-fonte 14 - Demonstração de treinamento do Word2Vec usando Gensim.....39
Código-fonte 15 - Demonstração do uso Word2Vec com spaCy 41
Código-fonte 16 - Demonstração de operação vetorial com embedding do Word2Vec 42
Código-fonte 17 - Carregando dataset SPAM e pré-processando com spaCy 46
Código-fonte 18 - Transformando dataset SPAM com BoW 47
Código-fonte 19 - Classificador Naive-Bayes aplicado sobre o dataset SPAM vetorizado por BoW.....48
Código-fonte 20 - Análise de Sentimentos com NLTK VADER.....50
Código-fonte 21 - Redução de dimensão do BoW do classificador de texto por método wrapper 52
Código-fonte 22 - Redução de dimensão do BoW com método não supervisionado em cascata, Truncated SVD (LSA), seguido de T-SNE 54
Código-fonte 23 - Agrupamento usando DBSCAN sobre os vetores T-SNE 56
Código-fonte 24 - Mostrando as 10 primeiras frases para um grupo selecionado 57

PREVIEW

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Instalação do Sklearn, Pandas e NumPy 9
Comando de prompt 2 - Instalação do Matplotlib e Seaborn 11
Comando de prompt 3 - Instalação do Gensim.....30
Comando de prompt 4 - Download do modelo em inglês do spaCy.....45

FIAP

SUMÁRIO

1 NLP NO ESTILO CLÁSSICO: ESTATÍSTICA, VETORES E EMOÇÕES EM TEXTO 6
1.1 O que é a abordagem estatística? 6
1.2 Quais são as aplicações de sistemas estatísticos em NLP? 6
1.3 Aprendizado clássico 7
1.4 Elementos de estatística descritiva 10
1.5 Modelos de probabilidade 13

2 MODELOS PROBABILÍSTICOS DE LINGUAGEM 19
2.1 N-gram 19
2.2 HMM 23
2.3 LDA 29

3 REPRESENTAÇÃO VETORIAL DE TEXTO 32
3.1 OHE 32
3.2 BoW 33
3.3 TF-IDF 35
3.4 Word2Vec 37

4 APRENDIZADO CLÁSSICO APLICADO À NLP 44
4.1 Downstream Task 44
4.2 Classificação 45
4.3 Redução de dimensionalidade e agrupamento 51

REFERÊNCIAS 59

FIAP

1 NLP NO ESTILO CLÁSSICO: ESTATÍSTICA, VETORES E EMOÇÕES EM TEXTO

1.1 O que é a abordagem estatística?

Modelos estatísticos em Processamento de Linguagem Natural (NLP) são algoritmos que utilizam técnicas baseadas em probabilidade e estatística para lidar com a linguagem humana de forma automatizada. Esses modelos analisam grandes volumes de dados linguísticos já observados — como corpora de textos — para identificar padrões recorrentes, relações entre palavras e estruturas sintáticas. Com base nessas análises, tornam-se capazes de realizar tarefas como previsão de palavras, análise de sentimentos, tradução automática e classificação de textos.

Em vez de entender o significado das palavras de forma semântica, esses modelos operam com base na frequência com que determinados elementos aparecem juntos ou em sequência, estimando probabilidades que guiam decisões futuras. Por exemplo, ao calcular a probabilidade de uma palavra ocorrer após outra, modelos como os de n-gramas conseguem gerar ou corrigir frases baseados em padrões estatísticos do idioma. Esses modelos foram fundamentais na segunda onda de desenvolvimento do NLP e continuam sendo utilizados, especialmente em tarefas que possuem recursos computacionais ou dados de treinamento limitados.

1.2 Quais são as aplicações de sistemas estatísticos em NLP?

A abordagem estatística em NLP é amplamente aplicada em tarefas nas quais o volume de dados, a ambiguidade linguística e a variabilidade contextual dificultam a construção de regras fixas. Esses sistemas se destacam por aprenderem padrões a partir de grandes corpora de texto, utilizando técnicas baseadas em probabilidade e frequência.

Alguns exemplos são:

- **Classificadores de texto:** como filtros de spam, detectores de discurso de ódio ou sistemas de análise de sentimentos. Utilizam modelos como Naive Bayes, regressão logística ou máquinas de vetor de suporte (SVM),

treinados a partir de textos anotados, para prever categorias com base em representações vetoriais, como TF-IDF ou BoW.

- **Modelos de linguagem baseados em n-grams:** utilizados em tarefas como correção automática, sugestões de palavras e reconhecimento de fala. Esses modelos estimam a probabilidade de uma palavra ocorrer com base nas anteriores, sendo úteis em aplicações com restrições computacionais.
- **Tradução automática baseada em modelos estatísticos:** antes do avanço das redes neurais, sistemas como o Google Translate utilizavam grandes corpora paralelos para estimar probabilidades de tradução entre segmentos de texto, baseando-se em modelos como IBM Models e alignments.
- **Sistemas de recuperação de informação e busca textual:** onde a relevância de documentos frente a uma consulta é estimada com base em medidas estatísticas como TF-IDF e BM25, em vez de regras semânticas.
- **Extração de informações estatística:** aplicada em mineração de texto, por exemplo, na identificação de padrões frequentes de coocorrência entre palavras ou entidades, como nomes de pessoas e locais, mesmo sem regras fixas ou dicionários.

1.3 Aprendizado clássico

O aprendizado de máquina clássico refere-se a um conjunto de algoritmos supervisionados e não supervisionados que aprendem padrões a partir de dados estruturados, sem recorrer a redes neurais profundas. Entre os algoritmos mais comuns estão regressão logística, Naive Bayes, máquinas de vetor de suporte, árvores de decisão e k-vizinhos mais próximos (k-NN).

Esses métodos se destacam por serem eficientes em contextos com dados limitados, interpretáveis e relativamente simples de treinar. No contexto de NLP, esses algoritmos são aplicados em tarefas como classificação de texto, análise de sentimentos, detecção de spam e extração de tópicos, nas quais o texto precisa ser

representado numericamente — geralmente por meio de vetores esparsos, como os gerados por Bag of Words (BoW), TF-IDF ou n-grams.

A relação entre aprendizado de máquina clássico e modelos estatísticos em NLP é direta e complementar. Os modelos estatísticos fornecem as bases teóricas e representações que transformam texto em dados estruturados, como frequências de palavras, distribuições de probabilidade e coocorrências, que, por sua vez, alimentam os algoritmos de aprendizado.

Por exemplo, um modelo TF-IDF gera vetores ponderados que destacam palavras relevantes em documentos, permitindo que um classificador como SVM separe as categorias baseando-se nesses padrões. Da mesma forma, o Naive Bayes — um modelo probabilístico clássico — é, ao mesmo tempo, um algoritmo de aprendizado e um modelo estatístico, sendo amplamente utilizado em NLP devido à sua simplicidade e eficácia.

Primeiramente, é importante estarmos familiarizados com as bibliotecas de aprendizado de máquina em Python antes de nos aprofundarmos no uso do aprendizado clássico com modelos estatísticos.

No decorrer deste capítulo, utilizaremos a biblioteca Scikit-Learn, pois ela fornece uma interface simples, eficiente e consistente para uma grande variedade de algoritmos supervisionados e não supervisionados — como regressão, classificação, *clustering*, redução de dimensionalidade e seleção de atributos.

No contexto de NLP, a Scikit-Learn é utilizada em conjunto com vetorizadores como CountVectorizer e TfidfVectorizer, que transformam textos em representações numéricas para alimentar modelos como Naive Bayes, SVM ou árvores de decisão.

Além disso, a biblioteca oferece ferramentas para pipeline de processamento, validação cruzada, ajuste de hiperparâmetros e avaliação de desempenho, tornando-se uma escolha ideal para construir protótipos de sistemas de NLP baseados em métodos estatísticos e de aprendizado de máquina tradicional.

Vejamos um exemplo de uso do aprendizado clássico em Python aplicado a um dataset numérico. Inicialmente, é preciso garantir que a biblioteca `sklearn` esteja instalada. Também será necessário instalar as bibliotecas `Pandas`, útil para o trabalho

com tabelas, e `NumPy`, voltada para a manipulação de vetores e matrizes — ambas amplamente utilizadas no contexto de aprendizado de máquina.

```
pip install scikit-learn pandas numpy
```

Comando de prompt 1 - Instalação do Sklearn, Pandas e NumPy
Fonte: Elaborado pelo autor (2025)

Uma vez que as bibliotecas estejam instaladas, podemos executar o código-fonte “Aprendizado de Máquina Clássico com Scikit-Learn em Python”, que utiliza o dataset íris do próprio sklearn — este é um dataset sobre botânica.

O objetivo deste código é carregar os dados e, em seguida, separar as colunas que servirão como entrada do modelo (variáveis independentes) daquelas que serão a saída do modelo (variáveis dependentes). Neste caso, a variável dependente é uma das três possíveis classes de flores presentes no dataset. Separamos os dados de treinamento e teste do modelo de aprendizado de máquina, e treinamos o modelo apenas com os dados de treinamento devidamente escalonados. Por fim, realizamos novas predições utilizando as entradas de teste escalonadas, a fim de comparar os resultados com o gabarito previamente separado na variável `y_test`. A acurácia do modelo e a matriz de confusão são mostradas como resultado.

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. Carregar o dataset Iris
iris = datasets.load_iris()
X = iris.data  # Variáveis independentes
y = iris.target  # Variável dependente (classe)

# 2. Converter para um DataFrame do pandas
df = pd.DataFrame(X, columns=iris.feature_names)
df['species']=pd.Categorical.from_codes(y, iris.target_names)

# Visualizando as primeiras linhas do DataFrame
df.head()

# 3. Separar os dados em treino e teste (80% treino, 20% teste)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
# 4. Normalização dos dados com StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 5. Criar e treinar o modelo de regressão logística
model = LogisticRegression(max_iter=200)
model.fit(X_train_scaled, y_train)

# 6. Fazer previsões
y_pred = model.predict(X_test_scaled)

# 7. Avaliar o modelo
accuracy = accuracy_score(y_test, y_pred)
conf_matrix = confusion_matrix(y_test, y_pred)

print(f"Acurácia do modelo: {accuracy * 100:.2f}%")
print("Matriz de Confusão:")
print(conf_matrix)
```

Código-fonte 1 - Aprendizado de máquina clássico com Scikit-Learn em Python
Fonte: Elaborado pelo autor (2025)

1.4 Elementos de estatística descritiva

A estatística descritiva é uma área da estatística que se dedica a coletar, organizar, resumir e apresentar os dados de forma clara e informativa. Seus principais elementos incluem as medidas de tendência central (como média, mediana e moda), que indicam o ponto em torno do qual os dados se distribuem; as medidas de dispersão (como desvio padrão, variância e amplitude), que mostram o grau de variação dos dados; além das medidas de posição (como quartis e percentis), que dividem os dados em partes iguais para facilitar a análise.

Outro componente importante é a representação gráfica dos dados, por meio de histogramas, diagramas de caixa, gráficos de barras e outros, que ajudam a visualizar padrões, distribuições e possíveis outliers. Esses elementos permitem uma compreensão inicial dos dados antes de se aplicar técnicas inferenciais mais complexas.

Frequentemente, a realização de tarefas de aprendizado de máquina exige uma etapa prévia de análise de dados, utilizando elementos da estatística descrita, conforme discutido anteriormente. Mais do que isso, essas ferramentas podem ser úteis para montar predições acerca de como os dados se comportarão no futuro. Aqui,

abordaremos apenas os conceitos de histograma e distribuição, que serão úteis para as aplicações de NLP que estudaremos adiante.

Um histograma é um tipo de gráfico de barras utilizado na estatística descritiva para representar a distribuição de frequências de um conjunto de dados numéricos contínuos. Ele mostra como os dados estão distribuídos ao longo de diferentes intervalos, com a altura de cada barra indicando a frequência (absoluta ou relativa) de observações dentro daquele intervalo.

Com o mesmo conjunto de dados do código-fonte “Aprendizado de Máquina Clássico com Scikit-Learn em Python”, vamos gerar os gráficos de histograma e de distribuição. Para isso, é necessário garantir que as bibliotecas de geração de gráfico estejam instaladas no Python, conforme o comando a seguir:

```
pip install matplotlib seaborn
```

Comando de prompt 2 - Instalação do Matplotlib e Seaborn
Fonte: Elaborado pelo autor (2025)

Agora, podemos executar o código-fonte “Análise exploratória e estatística descritiva em Python”. Neste código, estamos importando as bibliotecas de geração de gráficos, os dados do íris dataset do sklearn e a biblioteca Pandas para trabalhar com tabelas. Apenas geramos o gráfico de histograma com o comando `sns.histplot`.

O primeiro parâmetro da função é a variável que iremos plotar, o parâmetro `kde=True` faz com que além do gráfico de barras do histograma, também seja plotada uma linha contínua que representa a distribuição estimada dos dados.

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# Escolher duas variáveis para análise
var1 = iris.feature_names[0]
var2 = iris.feature_names[1]

# Criar subplots com histogramas e distribuições
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.histplot(df[var1], kde=True, ax=axes[0], color='skyblue')
axes[0].set_xlabel(var1)
axes[0].set_ylabel('Frequência')

sns.histplot(df[var2], kde=True, ax=axes[1], color='salmon')
axes[1].set_xlabel(var2)
axes[1].set_ylabel('Frequência')

plt.tight_layout()
plt.show()
```

Código-fonte 2 - Análise exploratória e estatística descritiva em Python
Fonte: Elaborado pelo autor (2025)

O resultado do código-fonte é mostrado na figura:

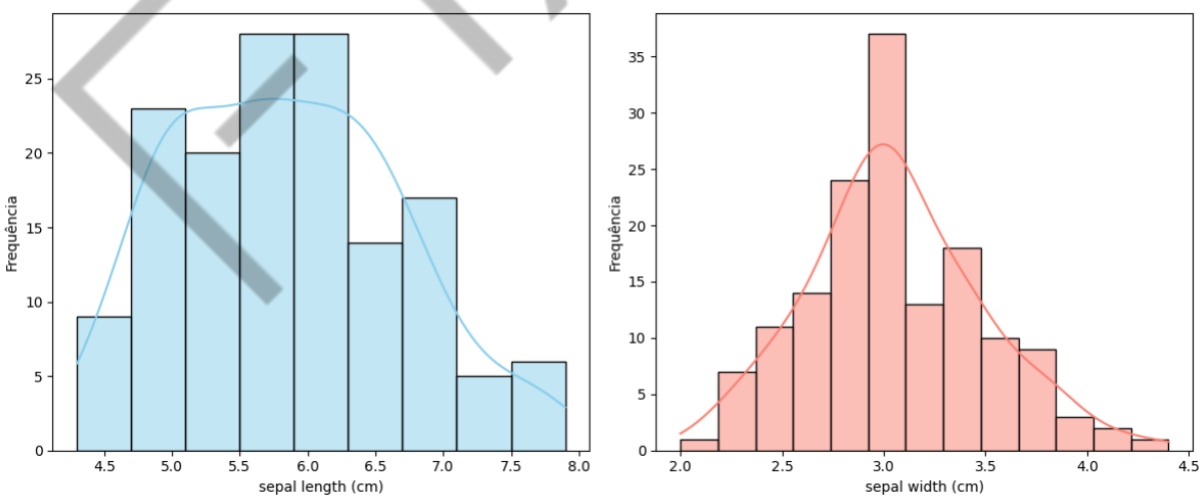


Figura 1 - Saída do código-fonte 2, histograma e distribuição
Fonte: Elaborado pelo autor (2025)

A relação do histograma com uma distribuição está no fato de que ele fornece uma representação visual da forma da distribuição dos dados — por exemplo, se ela é simétrica, assimétrica, normal (em forma de sino), uniforme ou se apresenta vários

picos (modos). Dessa forma, o histograma ajuda a identificar padrões, tendências e até possíveis anomalias (como outliers) nos dados. Quando bem construído, ele pode ser uma aproximação visual da **função densidade de probabilidade (PDF)** de uma distribuição estatística.

A função densidade de probabilidade é uma função matemática utilizada para descrever a probabilidade relativa de uma variável contínua assumir certos valores. Ao contrário de frequências absolutas (como em um histograma), a PDF mostra a densidade de probabilidade, e a área sob a curva em um intervalo representa a probabilidade de um valor estar dentro daquele intervalo. Por exemplo, para uma distribuição normal, a PDF é aquela clássica curva em formato de sino (*bell curve*).

Existem algumas formas práticas de avaliar o quão bem um histograma se aproxima da função PDF. Uma abordagem simples é a comparação visual, sobrepondo à representação do histograma uma curva KDE (Kernel Density Estimation), que corresponde a uma estimativa suave da PDF com base nos dados. Se a forma da KDE for semelhante à do histograma, isso sugere uma boa aproximação. Outra maneira é por meio de testes de aderência (*goodness-of-fit*), como os testes de Kolmogorov-Smirnov, Anderson-Darling ou Qui-quadrado, que comparam a distribuição empírica dos dados com uma distribuição teórica (por exemplo, a normal). Esses testes fornecem medidas estatísticas que indicam o grau de diferença entre a amostra observada e a distribuição esperada.

1.5 Modelos de probabilidade

Além do aprendizado de máquina e dos conceitos de histograma e distribuição, é fundamental ter domínio prévio dos conceitos de probabilidade antes de estudarmos as técnicas estatísticas de NLP.

Eventos e probabilidade são conceitos essenciais da estatística e da teoria das chances. Um evento é qualquer resultado ou conjunto de resultados possíveis de um experimento aleatório, como tirar cara ao lançar uma moeda. A probabilidade é a medida numérica da chance de um evento ocorrer, variando de 0 (impossível) a 1 (certo). Denotamos a probabilidade de um determinado evento A acontecer como $P(A)$. Para quantificar essa quantidade, normalmente contamos a quantidade de vezes que o evento A ocorreu e dividimos por todos os eventos que ocorreram,

inclusive aqueles diferentes de A. Por exemplo, ao lançar uma moeda temos dois eventos possíveis, $A = cara$ e $B = coroa$.

Vamos realizar um experimento: lancemos uma moeda 10 vezes e contemos quantas vezes deu cara. Supondo que a moeda seja justa (isto é, não viciada), os dois resultados possíveis — cara (A) e coroa (B) — têm a mesma probabilidade de ocorrer $P(A) = P(B) = 0{,}5$.

Cada lançamento de moeda é um exemplo clássico de uma **distribuição de Bernoulli**, que modela experimentos com apenas dois resultados possíveis: sucesso (por exemplo, obter cara) e fracasso (não obter cara, ou seja, obter coroa). A distribuição de Bernoulli tem um único parâmetro: p, a probabilidade de sucesso. No caso da moeda justa, $p = 0{,}5$. Assim, cada lançamento é uma variável aleatória de Bernoulli, e se repetirmos esse experimento várias vezes, obtemos uma soma de variáveis de Bernoulli, que segue uma distribuição binomial. Isso nos permite, com um número maior de lançamentos, aplicar conceitos como média, variância e até usar aproximações normais com base no Teorema Central do Limite.

O código-fonte “Exemplo de distribuições estatísticas” ilustra o Teorema Central do Limite, que afirma que, ao realizar muitos experimentos com uma variável aleatória — neste caso, a proporção de caras em lançamentos de moeda —, a distribuição das médias dessas amostras tende à distribuição normal à medida que o número de lançamentos (tamanho da amostra) aumenta, mesmo que a variável original (cara ou coroa) não siga uma distribuição normal. Isso é evidenciado pelos histogramas que, com mais lançamentos por experimento, se aproximam cada vez mais da curva da distribuição normal teórica.

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import norm, bernoulli

# Função para simular os lançamentos
def simular_moeda(n_lancamentos, n_experimentos=1000):
    resultados = []
    for _ in range(n_experimentos):
        lancamentos=np.random.choice([0,1],size=n_lancamentos)
        proporcao_caras = np.mean(lancamentos)
        resultados.append(proporcao_caras)
    return resultados
```

```
fig, axes = plt.subplots(2, 2, figsize=(8, 6))
axes = axes.flatten()

p = 0.5
valores = [0, 1]
probs = [1 - p, p]
axes[0].bar(valores,probs,color='skyblue', edgecolor='black')
axes[0].set_xticks(valores)
axes[0].set_ylim(0, 1)
axes[0].set_title("Distribuição de Bernoulli (p=0.5)")
axes[0].set_xlabel("Resultado")
axes[0].set_ylabel("Probabilidade")
axes[0].set_xticklabels(['Coroa (0)', 'Cara (1)'])


experimentos = [10, 100, 500]
for i, n in enumerate(experimentos, start=1):
    dados = simular_moeda(n)

    # Parâmetros da normal para aproximação
    mu = 0.5
    sigma = np.sqrt((0.5 * 0.5) / n)

    sns.histplot(dados, bins=20, kde=False, stat='density',
ax=axes[i], color='skyblue', edgecolor='black')

    x = np.linspace(0, 1, 100)
    y = norm.pdf(x, mu, sigma)
    axes[i].plot(x, y, color='red', label='PDF Normal')

    axes[i].set_title(f'{n} Lançamentos por Experimento')
    axes[i].set_xlabel('Proporção de Caras')
    axes[i].set_ylabel('Densidade')
    axes[i].legend()

plt.tight_layout()
plt.show()
```

Código-fonte 3 - Exemplo de distribuições estatísticas
Fonte: Elaborado pelo autor (2025)

Perceba que a distribuição de probabilidade do lançamento de moeda segue uma curva de Bernoulli para probabilidade 0,5. No entanto, para descobrir que esse valor é exatamente 0,5, poderíamos realizar 1000 experimentos com diferentes quantidades de lançamentos de moedas.

No primeiro experimento, realizamos 10 lançamentos de moeda, obtendo algumas caras e outras coroas. Ao calcular a proporção de vezes em que saiu cara (isto é, o número de caras dividido por 10), obtivemos, por exemplo, 0,4 — ou seja,

40% dos 10 lançamentos resultaram em cara. Repetindo o experimento com outros 10 lançamentos, obtivemos 60% para cara; em uma terceira repetição, 20%; e, na quarta, 80%. Se realizarmos esse processo 1000 vezes, teremos o segundo gráfico da figura “Saída do código-fonte 3, distribuição do lançamento de uma moeda e de vários lançamentos de moedas”, no qual cada barra representa a quantidade de vezes que obtivemos uma determinada proporção de caras.

Agora, vamos modificar o experimento para que cada um tenha 100 lançamentos de moedas. Com isso, obtemos o terceiro gráfico da figura. Quando aumentamos para 500 lançamentos por experimento, obtemos o quarto gráfico. Observe que, à medida que aumentamos o número de lançamentos, a distribuição da proporção de cara obtida em 1000 repetições para cada experimento se aproxima do limite central com uma proporção exatamente igual a 0,5.

A curva vermelha representa o modelo matemático dessa distribuição PDF, conhecida como **Curva Normal** — também chamada de Gaussiana, em homenagem a Gauss, seu formulador, ou ainda de Curva de Sino, devido ao seu formato.

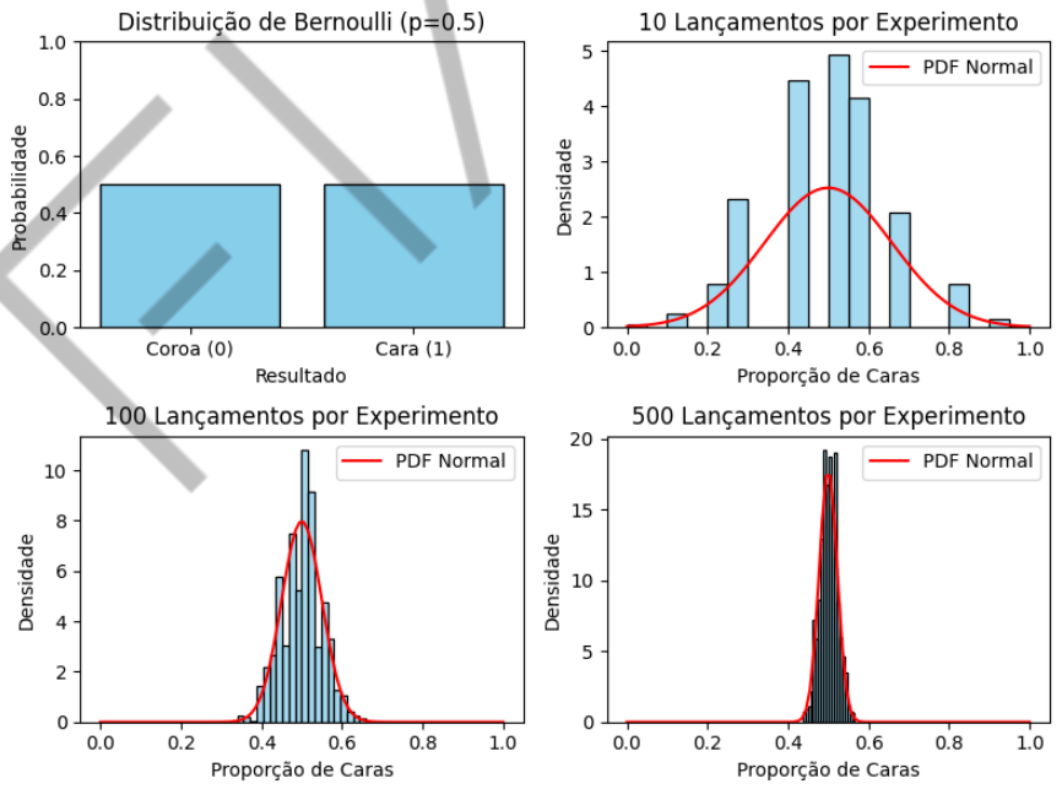


Figura 2 - Saída do código-fonte 3, distribuição do lançamento de uma moeda e de vários lançamentos de moedas
Fonte: Elaborado pelo autor (2025)

Em geral, a frequência das palavras segue distribuições de potência, como na Lei de Zipf. Logo, entender os fundamentos de probabilidade e estatística facilita a compreensão dos algoritmos que iremos discutir. Além da aproximação estatística para a determinação da função de distribuição de probabilidade, outros conceitos importantes de probabilidade estão relacionados à independência dos eventos.

A **independência entre eventos** significa que a ocorrência de um evento não afeta a probabilidade de ocorrência do outro. Dois eventos A e B são ditos independentes se, e somente se, a probabilidade de ambos ocorrerem simultaneamente for igual ao produto das probabilidades individuais, ou seja, $P(A \cap B) = P(A) \times P(B)$. Por exemplo, se lançarmos uma moeda e depois um dado, a probabilidade de obter cara na moeda ($P = 0{,}5$) e um 6 no dado ($P = 1/6$) é $P(cara\ e\ 6) = 0{,}5 \times 1/6 = 1/12$.

A independência é uma suposição crítica em muitos modelos estatísticos e algoritmos de aprendizado de máquina, pois a simplicidade do modelo depende diretamente da premissa de que as variáveis de entrada não influenciam umas às outras.

Se os eventos **não forem independentes**, isso significa que a ocorrência de um evento influencia a probabilidade de ocorrência do outro. Nesse caso, utilizamos o conceito de **probabilidade condicional**, que mede a chance de um evento ocorrer dado que outro já ocorreu. Matematicamente, dizemos que $P(A \mid B) \neq P(A)$, o que indica dependência entre A e B. Por exemplo, ao tirar duas cartas de um baralho sem reposição, o resultado da primeira tiragem afeta as probabilidades da segunda, pois o espaço amostral mudou. A presença de dependência entre eventos é comum em dados do mundo real e deve ser cuidadosamente tratada.

O **Teorema de Bayes** é um dos pilares da teoria da probabilidade e fornece uma forma de atualizar a probabilidade de um evento com base em novas evidências. Ele relaciona a probabilidade condicional de um evento A dado que B ocorreu com a probabilidade de B dado A, utilizando a fórmula:

$$P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}$$

Isso permite calcular a probabilidade a posteriori $P(A \mid B)$ a partir da probabilidade a priori $P(A)$, ajustada pela verossimilhança $P(B \mid A)$ e normalizada pela

probabilidade marginal de B $P(B)$. O teorema é amplamente utilizado em inferência estatística e aprendizado de máquina, especialmente em modelos como o classificador Naive Bayes, onde é possível fazer previsões mesmo com dados incompletos ou incertos, baseando-se em evidências observadas.

FIAP

2 MODELOS PROBABILÍSTICOS DE LINGUAGEM

2.1 N-gram

O modelo de n-gram em NLP é uma abordagem probabilística simples, porém poderosa, para modelar a linguagem com base em sequências de palavras. Um n-grama é uma sequência de n palavras consecutivas em um texto. Por exemplo, no trecho "o gato preto", os bigramas $(n = 2)$ são "o gato" e "gato preto".

Este modelo estima a probabilidade de uma palavra com base nas $(n - 1)$ palavras anteriores, ou seja, $P(palavra_{\mathrm{n}} \mid palavra_1, \ldots, palavra_{\mathrm{n-1}})$. Essa estimativa é geralmente feita contando a frequência das ocorrências no corpus. Por exemplo, um modelo de bigrama calcula $P(palavra_2 \mid palavra_1)$ baseado nas contagens de quantas vezes a sequência ocorreu.

Apesar de sua simplicidade, o modelo de n-gram enfrenta problemas como escassez de dados para sequências raras e falta de capacidade de capturar dependências de longo prazo. Ainda assim, foi amplamente utilizado em tarefas como correção automática, reconhecimento de fala e tradução automática, antes do avanço dos modelos neurais.

O modelo de n-gram está diretamente relacionado à hipótese de Markov, que assume que o futuro estado de um sistema depende apenas de um número limitado de estados anteriores — e não de toda a sua história passada. No contexto de NLP, isso significa que a probabilidade de uma palavra em uma sequência depende apenas das (n−1) palavras anteriores, e não do texto completo.

Essa é uma forma prática de lidar com a complexidade da linguagem natural, reduzindo o espaço de busca e tornando o problema computacionalmente viável. Por exemplo, em um modelo trigram (n=3), usamos a hipótese de Markov de ordem 2, assumindo que $P(palavra_{\mathrm{n}} \mid palavra_1, \ldots, palavra_{\mathrm{n-1}}) \approx P(palavra_{\mathrm{n}} \mid palavra_{\mathrm{n-2}}, palavra_{\mathrm{n-1}})$. Essa simplificação permite construir modelos estatísticos eficientes, embora com a limitação de ignorar dependências linguísticas de longo prazo que modelos mais avançados, como os baseados em redes neurais, conseguem capturar melhor.

A equação fundamental do modelo n-gram é a seguinte:

$$P(w_1, w_2, \ldots, w_n) \approx \prod_{i=1}^{n} P(w_i | w_{i-(n-1)}, \ldots, w_{i-1})$$

Essa fórmula expressa a probabilidade conjunta de uma sequência de palavras $w_1, w_2, \ldots w_n$, assumindo a hipótese de Markov de ordem (n−1) — ou seja, que cada palavra depende apenas das (n−1) palavras anteriores.

Exemplo com bigram (n=2):

$$P(w_1, w_2, \ldots, w_n) \approx P(w_1)P(w_2|w_1)P(w_3|w_2)$$

Na prática, a probabilidade condicional é estimada por contagem:

$$P(w_i|w_{i-1}) = \frac{Countagem(w_{i-1}, w_i)}{Contagem(w_i)}$$

Essa abordagem simples, porém eficaz, torna possível prever palavras ou pontuar sentenças em diversas aplicações, como corretores ortográficos, autocompletar e reconhecimento de fala.

A seguir, mostraremos como construir um modelo n-gram utilizando Python:

```
from collections import defaultdict
import math

class NGramModel:
    def __init__(self, n):
        self.n = n
        self.ngram_counts = defaultdict(int)
        self.context_counts = defaultdict(int)
        self.vocab = set()

    def train(self, corpus):
        tokens = corpus.lower().split()
        tokens = ['<s>'] * (self.n - 1) + tokens + ['</s>']
        self.vocab.update(tokens)

        for i in range(len(tokens) - self.n + 1):
            ngram = tuple(tokens[i:i+self.n])
            context = ngram[:-1]
            self.ngram_counts[ngram] += 1
            self.context_counts[context] += 1

    def prob(self, ngram):
        context = ngram[:-1]
        count_ngram = self.ngram_counts[ngram]
        count_context = self.context_counts[context]
```

```
        # Lidando com eventos não vistos (sem suavização aqui)
        if count_context == 0:
            return 0.0
        return count_ngram / count_context

    def sentence_prob(self, sentence):
        tokens = ['<s>'] * (self.n - 1) + sentence.lower().split() + ['</s>']
        total_prob = 1.0
        for i in range(len(tokens) - self.n + 1):
            ngram = tuple(tokens[i:i+self.n])
            p = self.prob(ngram)
            if p == 0:
                return 0.0
            total_prob *= p
        return total_prob

    def log_sentence_prob(self, sentence):
        tokens = ['<s>'] * (self.n - 1) + sentence.lower().split() + ['</s>']
        log_prob = 0.0
        for i in range(len(tokens) - self.n + 1):
            ngram = tuple(tokens[i:i+self.n])
            p = self.prob(ngram)
            if p == 0:
                return float('-inf')  # log(0)
            log_prob += math.log2(p)
        return log_prob

    def sugerir_proximas(self, contexto_texto, top_k=3):
        contexto = contexto_texto.lower().split()
        if len(contexto) < self.n - 1:
            contexto = ['<s>'] * (self.n - 1 - len(contexto)) + contexto
        else:
            contexto = contexto[-(self.n - 1):]

        contexto_tuple = tuple(contexto)
        candidatos = []
        for palavra in self.vocab:
            ngram = contexto_tuple + (palavra,)
            prob = self.prob(ngram)
            if prob > 0:
                candidatos.append((palavra, prob))

        # Ordenar e retornar as top_k sugestões
        candidatos.sort(key=lambda x: x[1], reverse=True)
        return candidatos[:top_k]
```

Código-fonte 4 - Modelo n-gram implementado em Python
Fonte: Elaborado pelo autor (2025)

Este código possui algumas funcionalidades, como prever a probabilidade de uma determinada frase aparecer, bem como antecipar qual é a próxima palavra mais provável, dadas *n* palavras anteriores. Por exemplo, é possível utilizar o código-fonte “Modelo n-gram implementado em Python” para calcular a probabilidade de uma determinada frase ocorrer.

```
corpus = "o gato preto pulou o muro o gato correu"
modelo = NGramModel(n=2)
modelo.train(corpus)

sentenca = "o gato correu"
print("Probabilidade:", modelo.sentence_prob(sentenca))
print("Log-probabilidade:",
modelo.log_sentence_prob(sentenca))
```

Código-fonte 5 - Uso do n-gram para prever probabilidade de uma sentença
Fonte: Elaborado pelo autor (2025)

Perceba que, neste caso, estamos utilizando bigramas para determinar a probabilidade. O treino do modelo consiste no cálculo das probabilidades de ocorrência individual das palavras e das palavras dadas as condições anteriores. Vale destacar que nosso corpus de treinamento é bastante reduzido, como se pode notar.

Por outro lado, podemos querer saber a próxima palavra de uma frase:

```
corpus = "o gato preto pulou o muro o gato correu"
modelo = NGramModel(n=2)
modelo.train(corpus)

sugestoes = modelo.sugerir_proximas("o", top_k=3)
print("Sugestões:", sugestoes)
```

Código-fonte 6 - Uso do n-gram para prever a próxima palavra
Fonte: Elaborado pelo autor (2025)

O resultado desse código é “Sugestões: `[('gato', 0.6666666666), ('muro', 0.3333333333)]`”. Isso significa que há 66,6% de chance da palavra “gato” vir na sequência da palavra “o”, e 33,3% de chance da palavra “muro” ocorrer na sequência. Como o corpus de treinamento é pequeno, podemos rastrear o motivo disso: a palavra “o” aparece três vezes, duas dela na frente da palavra “gato” e uma na frente da palavra “muro”. Por isso, o modelo de n-gram sugere essas duas possibilidades.

Podemos treinar com um corpus maior da seguinte forma:

```
with open('corpus.txt', 'r') as file:
    corpus = file.read()
modelo = NGramModel(n=2)
modelo.train(corpus)

sugestoes = modelo.sugerir_proximas("o", top_k=3)
print("Sugestões:", sugestoes)
```

Código-fonte 7 - Uso do n-gram para prever a próxima palavra usando corpus
Fonte: Elaborado pelo autor (2025)

No arquivo `.txt` de corpus carregado, foi utilizado o artigo da Wikipédia em português sobre Processamento de Linguagem Natural. O resultado é “Sugestões: `[('que', 0.05063291139240506), ('projeto', 0.05063291139240506), ('tipo', 0.0379746835443038)]`”.

Agora, as palavras “que”, “projeto” e “tipo” têm maior probabilidade de aparecer na sequência de “o” devido à natureza do corpus usado. Quanto maior for o corpus e mais próximo do domínio de aplicação, mais preciso será o resultado.

Importante: nos corpora de treinamento, normalmente são utilizados os tokens especiais para indicar começo <s> e fim </s>. Isso permite que o modelo de probabilidade condicional considere a probabilidade de um texto prosseguir de determinada forma assim que começa (iniciar com token <s>), ou que um texto que está sendo gerado deve ser terminado ao prever o token </s>.

2.2 HMM

Um **Modelo de Cadeia de Markov Oculta** (em inglês, **Hidden Markov Model — HMM**) é um modelo estatístico utilizado para representar sistemas que evoluem ao longo do tempo de forma probabilística, em que o estado real do sistema (chamado estado oculto) não é diretamente observável, mas pode ser inferido por meio de observações visíveis.

O HMM é amplamente utilizado em NLP para tarefas que envolvem sequências, como etiquetagem gramatical (POS tagging), reconhecimento de fala e análise de entidades nomeadas. Esse modelo assume que existe uma sequência subjacente de estados ocultos — por exemplo, categorias gramaticais — que gera uma sequência observável, como palavras em uma frase.

A estrutura do HMM é baseada em duas distribuições principais: a **probabilidade de transição** entre estados e a **probabilidade de emissão** de

observações a partir desses estados. Apesar de sua simplicidade e limitações — como a suposição de dependência apenas do estado anterior e observações condicionadas unicamente ao estado atual —, os HMMs capturam bem padrões estatísticos em dados sequenciais.

Na próxima figura, é ilustrado um esquemático de uma Cadeia de Markov. Temos dois estados denotados por “E” e por “A”. As setinhas laranjas representam transições entre estados: por exemplo, a setinha que sai de “A” e retorna para “A” representa a probabilidade do estado “A” ser atualizado para o mesmo estado (não ocorrer nada) e tem valor de 0,6 (60%); já a setinha que sai de “A” e vai para “E” representa a probabilidade de o estado “A” mudar para um estado “E”, que, neste caso, é de 0,4 (40%).

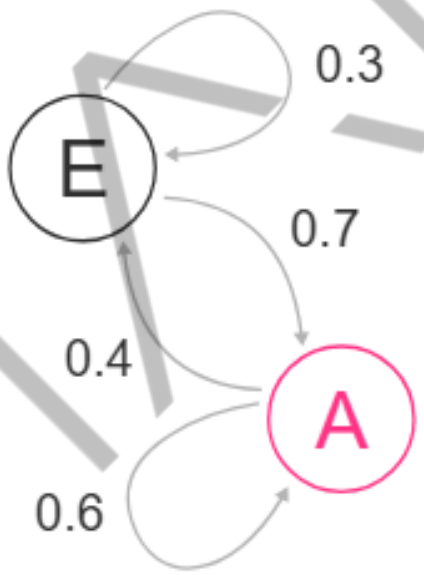


Figura 3 - Exemplo de uma Cadeia de Markov
Fonte: Google imagens (2025)

Uma Cadeia de Markov Oculta nada mais é do que uma Cadeia de Markov na qual os estados que conseguimos observar são somente uma parte de todos os estados possíveis. Na figura “Exemplo de uma Cadeia de Markov Oculta”, isso é representado pelas letras “S” e “C”, sendo “S” para estados que podemos observar e “C” para estados ocultos.

Quando observamos uma sequência de estados S1, S2, S3, as probabilidades de transições na verdade estão associadas aos estados ocultos C1, C2, C3, já que a mudança de estados ocorre entre eles e não nos estados observados. Isso faz surgir

o conceito de probabilidade de emissão, que é a chance de um determinado estado “S” ser visto quando o estado oculto “C” está vigente (na figura, são as setinhas que conectam o estado “C” com o “S”).

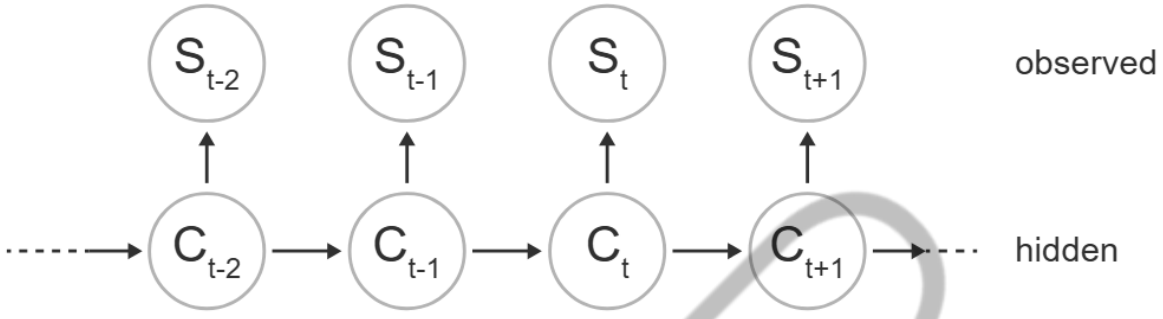


Figura 4 - Exemplo de uma Cadeia de Markov Oculta
Fonte: Google imagens (2025)

Agora que entendemos os modelos de Markov, aplicaremos esses conceitos na construção de um algoritmo de NER. Desta vez, não utilizaremos nenhum elemento de abordagem simbólica, como Regex e Gramáticas Livres de Contexto. Para isso, será necessário um corpus anotado, a partir do qual o algoritmo poderá calcular as probabilidades de cada palavra aparecer.

Vejamos a codificação central no código-fonte a seguir:

```
import numpy as np
from collections import defaultdict
import pandas as pd

class HMM_NER:
    def __init__(self):
        self.states = []
        self.words = []
        self.state_to_idx = {}
        self.word_to_idx = {}
        self.start_prob = None
        self.trans_prob = None
        self.emit_prob = None

    def fit(self, corpus):
        # Coletar estados e palavras únicas
        state_counts = defaultdict(int)
        start_counts = defaultdict(int)
        trans_counts = defaultdict(lambda: defaultdict(int))
        emit_counts = defaultdict(lambda: defaultdict(int))
        all_states = set()
        all_words = set()
```

```
        for sentence in corpus:
            prev_state = None
            for i, (word, state) in enumerate(sentence):
                all_states.add(state)
                all_words.add(word)
                state_counts[state] += 1
                emit_counts[state][word] += 1
                if i == 0:
                    start_counts[state] += 1
                if prev_state is not None:
                    trans_counts[prev_state][state] += 1
                prev_state = state

        self.states = sorted(list(all_states))
        self.words = sorted(list(all_words))
        n_states = len(self.states)
        n_words = len(self.words)

        self.state_to_idx = {s: i for i, s in
enumerate(self.states)}
        self.word_to_idx = {w: i for i, w in
enumerate(self.words)}

        # Probabilidades iniciais
        total_starts = sum(start_counts.values())
        self.start_prob = np.array([start_counts[s] /
total_starts for s in self.states])

        # Probabilidades de transição
        self.trans_prob = np.zeros((n_states, n_states))
        for s_from in self.states:
            total = sum(trans_counts[s_from].values())
            for s_to in self.states:
                if total > 0:

self.trans_prob[self.state_to_idx[s_from],
self.state_to_idx[s_to]] = trans_counts[s_from][s_to] / total

        # Probabilidades de emissão
        self.emit_prob = np.zeros((n_states, n_words))
        for state in self.states:
            s_idx = self.state_to_idx[state]
            total = state_counts[state]
            for word in emit_counts[state]:
                w_idx = self.word_to_idx[word]
                self.emit_prob[s_idx, w_idx] =
emit_counts[state][word] / total

    def viterbi(self, obs_sequence):
        obs_idx = [self.word_to_idx.get(w, -1) for w in
obs_sequence]
```

```
        T = len(obs_sequence)
        N = len(self.states)

        viterbi = np.zeros((N, T))
        backpointer = np.zeros((N, T), dtype=int)

        for s in range(N):
            idx = obs_idx[0]
            emit_p = self.emit_prob[s, idx] if idx != -1 else
1e-6
            viterbi[s, 0] = self.start_prob[s] * emit_p
            backpointer[s, 0] = 0

        for t in range(1, T):
            for s in range(N):
                idx = obs_idx[t]
                emit_p = self.emit_prob[s, idx] if idx != -1
else 1e-6
                prob = viterbi[:, t-1] * self.trans_prob[:, s]
* emit_p
                viterbi[s, t] = np.max(prob)
                backpointer[s, t] = np.argmax(prob)

        best_last_state = np.argmax(viterbi[:, -1])
        best_path_prob = np.max(viterbi[:, -1])

        best_path = [best_last_state]
        for t in range(T-1, 0, -1):
            best_last_state = backpointer[best_last_state, t]
            best_path.insert(0, best_last_state)

        state_path = [self.states[i] for i in best_path]
        return state_path, best_path_prob

    def show_emission_matrix(self):
        return                    pd.DataFrame(self.emit_prob,
index=self.states, columns=self.words).round(2)

    def show_transition_matrix(self):
        return                   pd.DataFrame(self.trans_prob,
index=self.states, columns=self.states).round(2)
```

Código-fonte 8 - Implementação de módulo NER através de HMM em Python
Fonte: Elaborado pelo autor (2025)

O uso do código-fonte “Implementação de módulo NER através de HMM em Python” é apresentado no seguinte código:

```
corpus = [
    [('John', 'ENT'), ('lives', 'O'), ('in', 'O'), ('London',
'ENT')],
    [('Alice', 'ENT'), ('works', 'O'), ('in', 'O'), ('Berlin',
'ENT')],
    [('He', 'O'), ('likes', 'O'), ('Paris', 'ENT')],
]

# Treinar o modelo
model = HMM_NER()
model.fit(corpus)

# Testar nova frase
test_sentence = ['Paulo', 'lives', 'in', 'Brazil']
state_path, prob = model.viterbi(test_sentence)

print("Frase:", test_sentence)
print("Reconhecimento de Entidades:", state_path)
print("Probabilidade:", round(prob, 4))

# Mostrar matrizes aprendidas
print("\nMatriz de Emissão:")
print(model.show_emission_matrix())
print("\nMatriz de Transição:")
print(model.show_transition_matrix())
```

Código-fonte 9 - Demonstração do NER-HMM em Python
Fonte: Elaborado pelo autor (2025)

O resultado pode ser visto na figura:

```
Frase: ['Paulo', 'lives', 'in', 'Brazil']
Reconhecimento de Entidades: ['ENT', 'O', 'O', 'ENT']
Probabilidade: 0.0

Matriz de Emissão:
     Alice  Berlin    He  John  London  Paris    in  likes  lives  works
ENT    0.2     0.2  0.00   0.2     0.2    0.2  0.00   0.00   0.00   0.00
O      0.0     0.0  0.17   0.0     0.0    0.0  0.33   0.17   0.17   0.17

Matriz de Transição:
     ENT    O
ENT  0.0  1.0
O    0.5  0.5
```

Figura 5 - Resultado do código-fonte 9, uso do NER-HMM
Fonte: Elaborado pelo autor (2025)

Perceba que as palavras “Paulo” e “Brazil” não estavam no corpus de treinamento, mas o NER anotou como entidade (ENT) corretamente. Porém, a probabilidade está marcada como 0 já que a palavra não existe no vocabulário. Neste

caso, estamos com sorte: a transição de estados ocultos favorece a identificação correta. No entanto, se tentarmos iniciar a frase com uma palavra sem ser entidade (por exemplo, “It”), o modelo irá erroneamente assumir que é uma entidade.

Para lidar com palavras novas, precisamos assumir uma distribuição de probabilidade diferente de zero para cada novo token a ser processado — isso é chamado de suavização (*smoothing*). Nesse processo, atribui-se uma pequena probabilidade de emissão para cada token desconhecido; por exemplo: `emit = self.emit_prob[s, obs_index] if obs_index >= 0 else 1e-6`. Outra possibilidade é adicionar uma pequena probabilidade a um token especial <UNK>, que representa qualquer palavra nova.

Dica: no jargão da área, palavras fora do vocabulário são chamadas pelo acrônimo OOV, do inglês *Out-Of-Vocabulary*.

2.3 LDA

Latent Dirichlet Allocation (LDA) é uma técnica amplamente utilizada em NLP para modelagem de tópicos em grandes volumes de texto. Trata-se de um modelo probabilístico que tenta descobrir automaticamente quais são os temas (ou “tópicos”) existentes em um conjunto de documentos, assumindo que cada documento é composto por uma mistura de tópicos e que cada tópico é representado por uma distribuição de palavras.

Em termos práticos, o LDA identifica padrões de coocorrência de palavras nos textos e agrupa palavras que costumam aparecer juntas, inferindo que pertencem a um mesmo tema. Por exemplo, se em vários documentos as palavras "governo", "presidente" e "eleição" aparecem frequentemente juntas, o modelo pode identificar isso como um tópico relacionado à política.

O processo envolve dois níveis principais de aleatoriedade: a escolha de tópicos para cada documento e a escolha de palavras para cada tópico, sendo ambos controlados por **distribuições de Dirichlet**, que dão nome ao modelo. O LDA é especialmente útil em tarefas como organização de grandes coleções de textos, sumarização de conteúdos e análise de tendências temáticas, sendo uma ferramenta essencial para quem trabalha com grandes volumes de dados textuais.

Para fazer um exemplo prático do LDA, vamos instalar o Gensim. O Gensim é uma biblioteca Python especializada em modelagem de tópicos, vetores semânticos e processamento de linguagem natural com foco em grandes volumes de texto não estruturado. É uma das ferramentas mais populares para tarefas como LDA, Word2Vec, TF-IDF, entre outras.

```
pip install gensim
```

Comando de prompt 3 - Instalação do Gensim
Fonte: Elaborado pelo autor (2025)

No código-fonte “Demonstração do LDA usando Gensim”, mostramos o uso do Gensim para realizar LDA em textos de exemplo. Note que foi utilizado o spaCy para realizar um pré-processamento, filtrando pontuação, números e stopwords.

```
import spacy
from gensim import corpora, models
import pprint

# Carregar modelo do spaCy em português
nlp = spacy.load("pt_core_news_sm")

# Corpus de exemplo
documents = [
    "Inteligência artificial está transformando a indústria.",
    "Aprendizado de máquina é uma subárea da inteligência artificial.",
    "Muitos cientistas de dados usam Python para análise de dados.",
    "Análise de dados envolve estatística, programação e conhecimento de negócio.",
    "Modelos de linguagem como o GPT são baseados em redes neurais."
]

# Pré-processamento com spaCy
texts = []
for doc in documents:
    doc_spacy = nlp(doc)
    tokens = [
        token.lemma_.lower()
        for token in doc_spacy
        if not token.is_stop and not token.is_punct and token.is_alpha
    ]
    texts.append(tokens)

# Criar dicionário e bag-of-words
```

FINAL

```
dictionary = corpora.Dictionary(texts)
corpus = [dictionary.doc2bow(text) for text in texts]

# Treinar o modelo LDA
lda_model = models.LdaModel(corpus, num_topics=2,
id2word=dictionary, passes=10, random_state=42)

# Exibir os tópicos
pprint.pprint(lda_model.print_topics())
```

Código-fonte 10 - Demonstração do LDA usando Gensim
Fonte: Elaborado pelo autor (2025)

O modelo retorna que existem dois tópicos demonstrando o peso de 10 palavras para compor cada tópico:

```
[(0,
  '0.091*"artificial" + 0.091*"inteligência" + 0.055*"aprendizado" + '
  '0.055*"máquina" + 0.055*"subárea" + 0.055*"envolver" + 0.055*"negócio" + '
  '0.055*"conhecimento" + 0.055*"estatístico" + 0.055*"programação"'),
 (1,
  '0.108*"dado" + 0.064*"análise" + 0.063*"rede" + 0.063*"linguagem" + '
  '0.063*"gpt" + 0.063*"neural" + 0.063*"basear" + 0.063*"modelo" + '
  '0.063*"python" + 0.063*"usar"')]
```

Figura 6 - Resultado do código-fonte 10, uso do LDA
Fonte: Elaborado pelo autor (2025)

Note que, no código-fonte “Demonstração do LDA usando Gensim”, há uma transformação dos tokens (palavras) em um dicionário diferente, utilizando o comando `doc2bow`. Este é um exemplo de mudança de representação de texto, quando transformamos a representação de texto de strings para uma representação puramente numérica.

Na próxima seção, iremos discutir exatamente essas técnicas e as maneiras pelas quais representamos texto por vetores.

3 REPRESENTAÇÃO VETORIAL DE TEXTO

3.1 OHE

O **One-Hot Encoding (OHE)** é uma técnica utilizada em Machine Learning e NLP para representar palavras de maneira numérica. Nesse método, cada palavra é convertida em um vetor binário no qual apenas uma posição específica do vetor é marcada com o valor 1, enquanto todas as demais posições são preenchidas com 0.

O tamanho do vetor é igual ao número total de palavras no vocabulário, e cada palavra recebe uma posição única. Isso cria uma representação esparsa e binária das palavras, como apresentada na figura “Representação gráfica da execução do OHE”.



Figura 7 - Representação gráfica da execução do OHE
Fonte: Elaborado pelo autor (2025)

Em Python, podemos fazer:

```python
from sklearn.preprocessing import OneHotEncoder
import numpy as np

palavras = ['gato', 'cachorro', 'pássaro', 'gato', 'cachorro']

# Criando e ajustando o OneHotEncoder
encoder = OneHotEncoder(sparse_output=False)
palavras_reshaped = np.array(palavras).reshape(-1, 1)
ohe = encoder.fit_transform(palavras_reshaped)

# Exibindo os resultados
print("Vetor One-Hot Encoding:\n", ohe)

# Mostrando os nomes das palavras no vocabulário
print("Vocabulário (ordem das palavras):")
print(encoder.categories_[0])
```

Código-fonte 11 - Demonstração do OHE usando Sklearn
Fonte: Elaborado pelo autor (2025)

A figura a seguir apresenta o resultado obtido. Observe que o vocabulário total tem apenas três palavras:

```
Vetor One-Hot Encoding:
 [[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]
 [0. 1. 0.]
 [1. 0. 0.]]
Vocabulário (ordem das palavras):
['cachorro' 'gato' 'pássaro']
```

Figura 8 - Resultado do código-fonte 11, uso do OHE
Fonte: Elaborado pelo autor (2025)

3.2 BoW

O **Bag of Words (BoW)** é uma técnica de representação textual utilizada em Machine Learning e NLP que, ao contrário do One-Hot Encoding, não codifica palavras isoladas, mas sim sentenças ou documentos inteiros. No BoW, a ideia central é tratar o texto como um “saco” de palavras, em que a ordem e a gramática são ignoradas, e a ênfase está apenas nas palavras presentes e suas frequências de ocorrência.

Para cada sentença, o BoW cria um vetor em que cada posição representa a presença (com 0 e 1) ou a frequência (contagem de vezes) de uma palavra específica do vocabulário, sem considerar a ordem delas. Dessa forma, enquanto o OHE mapeia

cada palavra para uma posição única em um vetor binário, o BoW constrói uma representação vetorial que considera cada palavra que aparece em um conjunto de palavras (texto). Isso o torna mais adequado para tarefas de classificação ou análise de sentimentos, em que o conteúdo semântico da frase é mais importante do que a estrutura gramatical.

No próximo código-fonte, demonstramos o uso do BoW com sklearn em Python. De fato, o BoW é criado com a função `CountVectorizer`, em que o parâmetro `max_features=5` define apenas cinco palavras como vocabulário de transformação. Já o parâmetro `binary=True` estabelece que a contagem será binária — ou seja, 0 se a palavra não estiver na sentença e 1 se tiver. Você pode trocar esses valores para verificar o funcionamento do modelo, experimentando frases que contenham repetições de palavras.

O resultado da execução do código-fonte “Demonstração do BoW usando Sklearn” é mostrado na figura “Resultado do código-fonte 12, uso do BoW”: observe que cada vetor da matriz bag of words representa uma das frases de entrada; diferente do OHE, o número 1 pode aparecer para mais de uma dimensão (posição do vetor).

```
from sklearn.feature_extraction.text import CountVectorizer

sentencas = [
    "O gato está na casa",
    "O cachorro está no jardim",
    "O pássaro voa no céu",
    "O gato e o cachorro brincam juntos"
]

# Criando o vetorizador de Bag of Words
vectorizer = CountVectorizer(max_features=5, binary=True)

# Ajustando e transformando as sentenças em uma matriz de contagem
X = vectorizer.fit_transform(sentencas)

# Exibindo a matriz gerada
print("Matriz Bag of Words:")
print(X.toarray())

# Exibindo o vocabulário (palavras mapeadas)
print("Vocabulário (ordem das palavras):")
print(vectorizer.get_feature_names_out())
```

Código-fonte 12 - Demonstração do BoW usando Sklearn
Fonte: Elaborado pelo autor (2025)

```
Matriz Bag of Words:
[[0 0 1 1 0]
 [0 1 1 0 1]
 [0 0 0 0 1]
 [1 1 0 1 0]]
Vocabulário (ordem das palavras):
['brincam' 'cachorro' 'está' 'gato' 'no']
```

Figura 9 - Resultado do código-fonte 12, uso do BoW
Fonte: Elaborado pelo autor (2025)

Dica: para que o BoW capture mais valor semântico, você pode fazer um pré-processamento com spaCy ou NTLK para remover stopwords das sentenças.

3.3 TF-IDF

Dando continuidade à discussão sobre bag of words, uma limitação dessa técnica é que ela trata todas as palavras com a mesma importância, independentemente de sua frequência no texto ou no corpus como um todo. Para superar essa limitação, a técnica de **TF-IDF (Term Frequency-Inverse Document Frequency)** foi desenvolvida.

O TF-IDF é uma medida que ajusta a contagem de palavras do BoW, levando em consideração tanto a frequência da palavra em um documento específico (TF — Term Frequency) quanto a sua importância em relação ao conjunto total de documentos (IDF — Inverse Document Frequency). A frequência do termo (TF) indica o quão frequentemente uma palavra aparece em um documento, enquanto o IDF diminui o peso das palavras que ocorrem com frequência em muitos documentos — como é o caso de palavras comuns, por exemplo: "de", "o", "a".

Esse ajuste torna o modelo mais sensível a palavras mais exclusivas e informativas, sendo especialmente útil em tarefas como recuperação de informação, classificação de texto e análise de sentimentos — contextos em que a presença de palavras raras é mais significativa do que a de palavras comuns a todos os documentos.

Historicamente, o TF-IDF foi proposto como uma medida de relevância para sistemas de **recuperação de informação** (Information Retrieval — IR), visando avaliar a importância das palavras em documentos de maneira quantitativa e baseada

em seu contexto. A principal motivação para o desenvolvimento do TF-IDF foi melhorar a precisão da busca e recuperação de documentos que contenham informações relevantes para uma consulta, utilizando um modelo baseado em **similaridade vetorial**.

O cálculo do TF-IDF envolve duas etapas principais: calcular o TF e o IDF, e então multiplicar esses dois valores para obter o TF-IDF. A frequência do termo é uma medida de quantas vezes uma palavra aparece em um documento. Ou seja, o TF normaliza a frequência da palavra, dividindo o número de vezes que ela aparece pelo total de palavras no documento. Pode ser calculada de forma simples como:

$$TF(t,d) = \frac{Número\ de\ vezes\ que\ o\ termo\ \boldsymbol{t}\ aparece\ no\ documento\ \boldsymbol{d}}{Número\ total\ de\ termos\ no\ documento\ \boldsymbol{d}}$$

Já o IDF é uma medida de quão importante é uma palavra no contexto do corpus. Quanto mais rara a palavra no corpus, maior será o valor do IDF. A fórmula é:

$$IDF(t) = \log\left(\frac{Número\ total\ de\ documentos}{Número\ de\ documentos\ com\ o\ termo\ \boldsymbol{t}}\right)$$

A ideia por trás do IDF é que, se uma palavra aparece em muitos documentos, ela provavelmente não é muito informativa, enquanto se aparecer em poucos documentos, ela tem maior poder discriminativo.

Por fim, o valor final do TF-IDF é simplesmente o produto do TF e do IDF:

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

Dessa forma, cada dimensão (palavra do vocabulário) utilizada para representar a sentença de entrada (ou documento de texto como um todo), receberá um valor não inteiro (o TF-IDF calculado). No BoW, esse valor era 0 ou 1 para binário ou 0, 1, 2, 3, entre outros para contagem de vezes.

Vejamos isso em um exemplo prático utilizando Python com a biblioteca Sklearn. No código-fonte “Demonstração do TF-IDF usando Sklearn”, instanciamos o transformador TF-IDF com `TfidfVectorizer()` antes de aplicá-lo às sentenças.

```python
from sklearn.feature_extraction.text import TfidfVectorizer

# Exemplo de documentos
documentos = [
    "gato está na casa",
    "gato cachorro está no jardim",
    "pássaro voa no céu"
]

# Criando o vetorizador TF-IDF
vectorizer = TfidfVectorizer()

# Ajustando e transformando os documentos
X = vectorizer.fit_transform(documentos)

# Exibindo a matriz TF-IDF com 3 casas decimais
print("Matriz TF-IDF:")
print(np.round(X.toarray(), 3))

# Exibindo o vocabulário (palavras mapeadas)
print("Vocabulário (ordem das palavras):")
print(vectorizer.get_feature_names_out())
```

Código-fonte 13 - Demonstração do TF-IDF usando Sklearn
Fonte: Elaborado pelo autor (2025)

O resultado do código-fonte pode ser visto na próxima figura. Observe que os valores em cada elemento do vetor são números reais em vez de números inteiros, como no BoW.

```
Matriz TF-IDF:
[[0.    0.563 0.    0.428 0.428 0.    0.563 0.    0.    0.   ]
 [0.517 0.    0.    0.394 0.394 0.517 0.    0.394 0.    0.   ]
 [0.    0.    0.529 0.    0.    0.    0.    0.402 0.529 0.529]]
Vocabulário (ordem das palavras):
['cachorro' 'casa' 'céu' 'está' 'gato' 'jardim' 'na' 'no' 'pássaro' 'voa']
```

Figura 10 - Resultado do código-fonte 13, uso do TF-IDF
Fonte: Elaborado pelo autor (2025)

Observação: assim como no BoW, é possível passar o parâmetro `max_features` para controlar o tamanho do vocabulário sendo considerado.

3.4 Word2Vec

O **Word2Vec** é um modelo de aprendizado de máquina desenvolvido por pesquisadores do Google em 2013, com o objetivo de gerar representações vetoriais contínuas para palavras que preservam suas relações semânticas e contextuais. Em

vez de representar palavras como vetores esparsos e binários (como no One-Hot Encoding), o Word2Vec mapeia as palavras para vetores densos de dimensões fixas, nos quais palavras com significados semelhantes ou que aparecem em contextos semelhantes possuem representações vetoriais próximas no espaço vetorial.

O Word2Vec pode ser treinado utilizando duas abordagens principais: **Continuous Bag of Words (CBOW)**, que tenta prever uma palavra a partir do contexto ao seu redor; e **Skip-gram**, que tenta prever o contexto a partir de uma palavra central (a comparação entre esses modelos é apresentada na figura a seguir). Essa técnica permite que palavras com significados semelhantes, como "rei" e "rainha" ou "gato" e "cachorro", tenham representações vetoriais próximas, oferecendo uma representação semântica rica que é amplamente utilizada em diversas tarefas de NLP.

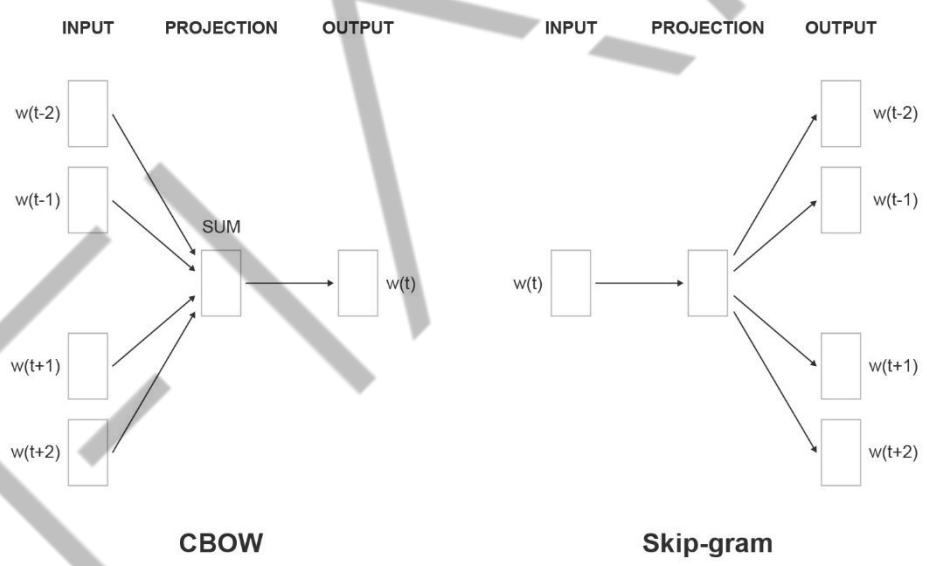


Figura 11 - Esquemas de treinamento do Word2Vec
Fonte: Google imagens (2025)

No **CBOW**, o modelo tenta prever uma palavra a partir de seu contexto, ou seja, usa as palavras ao redor de um termo alvo (o contexto) para prever a palavra central. Por exemplo, em uma sentença como "O gato está no jardim", o modelo tentaria prever a palavra "gato" a partir das palavras ao redor ("O", "está", "no", "jardim").

Já no **Skip-gram**, a abordagem é invertida: dada uma palavra central, o modelo tenta prever as palavras ao seu redor. Por exemplo, dada a palavra "gato", o modelo

tentaria prever as palavras "O", "está", "no" e "jardim". O modelo Skip-gram é especialmente útil quando se tem grandes quantidades de texto e se busca capturar palavras raras com alta precisão.

No código-fonte a seguir, apresentamos o funcionamento do Word2Vec, desde seu treinamento até sua aplicação prática.

```
import gensim
from gensim.models import Word2Vec

# Exemplo de um pequeno corpus de texto
corpus = [
    "o gato está no jardim",
    "o cachorro está no jardim",
    "o gato corre rápido",
    "cachorros são leais",
    "o pássaro voa alto",
    "gatos são independentes"
]

# Pré-processamento simples: Tokenizando as palavras
# Vamos separar as frases em listas de palavras
sentences = [sentence.split() for sentence in corpus]

# Treinando o modelo Word2Vec
model = Word2Vec(sentences, vector_size=10, window=3,
min_count=1, workers=4)

# Salvando o modelo treinado
model.save("word2vec.model")

# Carregando o modelo treinado
model = Word2Vec.load("word2vec.model")

# Consultando palavras mais semelhantes
similar_words = model.wv.most_similar("gato", topn=3)

print("Palavras mais semelhantes a 'gato':")
for word, similarity in similar_words:
    print(f"{word}: {similarity:.4f}")

# Obtendo o vetor para a palavra "gato"
gato_vector = model.wv['gato']
print("\nVetor para 'gato':")
print(gato_vector)
```

Código-fonte 14 - Demonstração de treinamento do Word2Vec usando Gensim
Fonte: Elaborado pelo autor (2025)

A similaridade entre vetores de palavras é calculada por meio de geometria analítica, com a regra da **Similaridade do Cosseno**:

$$similaridade(\overrightarrow{w_1}, \overrightarrow{w_2}) = \frac{\overrightarrow{w_1} \cdot \overrightarrow{w_2}}{\|\overrightarrow{w_1}\| \|\overrightarrow{w_2}\|} = \frac{\sum_{i=1}^{n} \overrightarrow{w_1}[i]\, \overrightarrow{w_2}[i]}{\sqrt{\sum_{i=1}^{n} \overrightarrow{w_1}[i]^2}\, \sqrt{\sum_{i=1}^{n} \overrightarrow{w_2}[i]^2}}$$

Nesse cálculo, $\overrightarrow{w_1}$ e $\overrightarrow{w_2}$ são os vetores das palavras w_1 e w_2 . A multiplicação $\overrightarrow{w_1} \cdot \overrightarrow{w_2}$ é denominada produto escalar, que consiste na multiplicação das componentes de cada vetor, par a par, seguida da soma desses produtos no final. Já $\|\overrightarrow{w_1}\|$ e $\|\overrightarrow{w_2}\|$ são as normas (módulos) dos vetores, calculadas pela raiz quadrada da soma dos quadrados de seus componentes, como no teorema de Pitágoras.

O resultado de execução do código-fonte “Demonstração de treinamento do Word2Vec usando Gensim” é ilustrado na figura a seguir.

```
Palavras mais semelhantes a 'gato':
voa: 0.2914
alto: 0.2756
está: 0.2495

Vetor para 'gato':
[-0.08157917  0.04495798 -0.04137076  0.00824536  0.08498619 -0.04462177
  0.045175   -0.0678696  -0.03548489  0.09398508]
```

Figura 12 - Resultado do código-fonte 14, uso do Word2Vec
Fonte: Elaborado pelo autor (2025)

Observe que o vetor para a palavra “gato” possui 10 dimensões (elementos de vetor). Esse valor foi definido durante o treinamento por meio do parâmetro `vector_size=10`. Além disso, as palavras semelhantes a “gato” são “voa”, “alto” e “está”, o que está muito distante do que esperaríamos em português. Isso ocorre porque utilizamos poucos dados de treinamento (apenas seis frases).

Podemos utilizar um modelo pré-treinado do spaCy. O modelo `pt_core_news_md` do spaCy, voltado para o português, utiliza vetores de palavras treinados com uma abordagem semelhante ao Word2Vec, baseada em modelos de embeddings contínuos. No entanto, diferentemente da versão `lg` (large), o `md` (medium) possui uma cobertura mais limitada do vocabulário com vetores pré-treinados, o que pode afetar a qualidade das similaridades calculadas — especialmente para palavras menos frequentes.

Esses vetores foram treinados em grandes corpora de texto utilizando algoritmos semelhantes ao Skip-Gram com Negative Sampling, como no Word2Vec original, mas adaptados à arquitetura interna do spaCy. Além disso, o modelo `md` inclui vetores de dimensão reduzida (geralmente 300 dimensões), o que oferece um bom equilíbrio entre desempenho e eficiência computacional. No entanto, isso sacrificaria um pouco a riqueza semântica quando comparado a modelos maiores, como o `lg`:

```
import spacy

nlp = spacy.load("pt_core_news_md")

palavra1 = nlp("rei")
palavra2 = nlp("rainha")
palavra3 = nlp("carro")

sim1 = palavra1.similarity(palavra2)  # rei vs rainha
sim2 = palavra1.similarity(palavra3)  # rei vs carro

print(f"Similaridade entre 'rei' e 'rainha': {sim1:.4f}")
print(f"Similaridade entre 'rei' e 'carro': {sim2:.4f}")
```

Código-fonte 15 - Demonstração do uso Word2Vec com spaCy
Fonte: Elaborado pelo autor (2025)

O resultado da execução desse código-fonte é: `Similaridade entre 'rei' e 'rainha': 0.6001 Similaridade entre 'rei' e 'carro': 0.0908`. Isso significa que “rei” e “rainha” são muito mais semelhantes do que “rei” e “carro”, o que era esperado.

Importante: no jargão de NLP, **embedding** é uma representação vetorial densa que captura o significado semântico de palavras em um espaço numérico.

Modelos de representação como o Word2vec possibilitam a realização de analogias linguísticas por meio da manipulação dos embeddings das palavras. Isso é feito com base na geometria analítica, através da soma e subtração de vetores. Por exemplo, peguemos as palavras “rei”, “homem” e “mulher”. Qual o resultado da seguinte operação vetorial: rei-homem+mulher? Do ponto de vista do português, essa é uma operação sem sentido, mas, matematicamente, gostaríamos que essa expressão retornasse uma nova palavra com algum significado semântico.

Observe o resultado:

```
Resultado da analogia 'rei - homem + mulher':
rei: 0.8917
rainha: 0.7387
princesa: 0.6005
mulher: 0.3644
homem: 0.1751
mesa: 0.1533
carro: 0.0385
```

Figura 13 - Resultado do código-fonte 16, operação vetorial com Word2Vec
Fonte: Elaborado pelo autor (2025)

Esse resultado é obtido por meio da operação no espaço vetorial, seguido do cálculo da similaridade do cosseno para uma lista de palavras. É possível expandir a lista de palavras para todo o vocabulário do modelo, truncando para resultados abaixo de certo limiar de similaridade.

Isso foi implementado em Python, observe:

```python
import spacy
from scipy import spatial

# Carrega o modelo pre-treinado
nlp = spacy.load("pt_core_news_md")

# Palavras a serem usadas na operacao vetorial
rei = nlp("rei").vector
homem = nlp("homem").vector
mulher = nlp("mulher").vector

# Operacao vetorial: rei - homem + mulher
vetor_resultante = rei - homem + mulher

# Lista de palavras de exemplo
palavras = ["rainha", "princesa", "rei", "homem", "mulher",
"carro", "mesa"]

# Calcula a similaridade entre o vetor resultante e os vetores
das palavras
mais_proxima = sorted(
    [(w, 1 - spatial.distance.cosine(nlp(w).vector,
vetor_resultante)) for w in palavras],
    key=lambda x: -x[1]
)

print("Resultado da analogia 'rei - homem + mulher':")
for palavra, score in mais_proxima:
    print(f"{palavra}: {score:.4f}")
```

Código-fonte 16 - Demonstração de operação vetorial com embedding do Word2Vec
Fonte: Elaborado pelo autor (2025)

Modelos como OHE, BoW, TF-IDF e Word2Vec permitem representar palavras e sentenças em vetores. Apesar da riqueza semântica capturada no algoritmo de Word2Vec permitir que esse modelo representacional seja usado para geração de texto e classificação semântica de palavras, tarefas mais complexas exigem de mais camadas de processamento.

Na próxima seção, estudaremos como aplicar técnicas de aprendizado de máquina sobre essas representações.

FIAP

4 APRENDIZADO CLÁSSICO APLICADO À NLP

4.1 Downstream Task

Downstream tasks são as tarefas que utilizam modelos de representação de texto como base para resolver problemas específicos de NLP. Essas tarefas geralmente dependem de representações de texto (como OHE, BoW, TF-IDF e Word2Vec) e são combinadas com técnicas de aprendizado de máquina clássico (como classificação, agrupamento e redução de dimensionalidade) para alcançar um objetivo específico, como prever rótulos, agrupar documentos semelhantes ou reduzir a complexidade dos dados.

O fluxo de trabalho típico começa com o pré-processamento do texto, transformando-o em uma representação numérica por meio de técnicas como OHE, BoW, TF-IDF ou Word2Vec — dependendo do caso de uso e da complexidade desejada. Em seguida, essa representação é alimentada em um modelo de aprendizado de máquina para a realização de uma downstream task, como classificação, agrupamento ou redução de dimensionalidade.

Na classificação, por exemplo, o objetivo é atribuir uma categoria (ou rótulo) a um texto, como no caso da classificação de sentimentos (positivo, negativo e neutro), categorização de tópicos ou identificação de spam. Após representar o texto com BoW, TF-IDF ou Word2Vec, utilizamos algoritmos de aprendizado de máquina clássico, como SVM, árvores de decisão ou redes neurais, para ensinar o modelo a associar as representações de texto aos rótulos.

O agrupamento (*clustering*) é outra downstream task, cujo objetivo é agrupar textos ou documentos semelhantes sem rótulos pré-definidos. Um exemplo seria agrupar artigos de notícias por tópico ou segmentar usuários com base em interações em texto. Para isso, também utilizamos as representações vetoriais de texto como BoW e TF-IDF e, em seguida, algoritmos de agrupamento como K-means ou DBSCAN para formar os grupos.

A redução de dimensionalidade é uma tarefa que visa reduzir o número de características representadas, mas preservando as informações mais importantes. Em modelos como BoW ou TF-IDF, que podem ter centenas de dimensões, técnicas

como PCA (Principal Component Analysis) ou t-SNE podem ser utilizadas para reduzir a dimensionalidade dos dados e facilitar outras tarefas, como visualização ou preparação para outros modelos.

4.2 Classificação

Classificação é a tarefa de aprendizado de máquina supervisionada na qual uma sequência de números (valores de features) são utilizados para inferir uma classe (inteiro ou string). Como downstream task, a classificação usa os **vetores de texto** OHE, BoW, TF-IDF ou Word2Vec como se fosse o **vetor de features** de entrada.

Para entender melhor esse conceito, vamos construir dois exemplos práticos que usam bibliotecas de aprendizado clássico envolvendo representação vetorial. O primeiro tem o objetivo de classificar se o texto é uma pergunta dissertativa ou optativa.

Dica: você pode conferir os códigos no asset do capítulo.

Já o objetivo do segundo exemplo a seguir é desenvolver um sistema de detecção de spam que analise o texto contido na mensagem. Para isso, precisamos ter um dataset anotado, contendo os devidos rótulos para mensagens fraudulentas (spam) e legítimas (ham).

Dica: este dataset pode ser baixado no link:

<https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip>.

Baixe e extraia o arquivo SMSSpamCollection para a mesma pasta onde estiver rodando o código em Python.

Como iremos trabalhar com dados em inglês, talvez seja preciso instalar o modelo com esse idioma no spaCy:

```
python -m spacy download en_core_web_sm
```

Comando de prompt 4 - Download do modelo em inglês do spaCy
Fonte: Elaborado pelo autor (2025)

Primeiramente, iremos carregar o dataset e realizar o pré-processamento com spaCy. Em seguida, faremos a análise léxica e morfológica, removendo stopwords, pontuação, números e lematizando as palavras restantes:

```
import pandas as pd
import spacy
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split

# Carregar modelo spaCy para inglês
nlp = spacy.load("en_core_web_sm")

# Carregar o dataset
df = pd.read_csv('smsspamcollection/SMSSpamCollection',
sep='\t', header=None, names=["Label", "Message"])

# Função de normalização do texto de entrada
def normalize_text(text):
    doc = nlp(text)
    tokens = [
        token.lemma_.lower()
        for token in doc
        if token.is_alpha and not token.is_stop
    ]
    return ' '.join(tokens)

# Criar nova coluna com texto normalizado
df['Message_Normalized']=df['Message'].apply(normalize_text)

# Visualizar o resultado
df.head()
```

Código-fonte 17 - Carregando dataset SPAM e pré-processando com spaCy
Fonte: Elaborado pelo autor (2025)

A figura “Resultado do código-fonte 17, carregando o dataset SPAM” mostra as cinco primeiras linhas do dataset como resultado da execução.

	Label	Message	Message_Normalized
0	ham	Go until jurong point, crazy.. Available only ...	jurong point crazy available bugis n great wor...
1	ham	Ok lar... Joking wif u oni...	ok lar joke wif u oni
2	spam	Free entry in 2 a wkly comp to win FA Cup fina...	free entry wkly comp win fa cup final tkts tex...
3	ham	U dun say so early hor... U c already then say...	u dun early hor u c
4	ham	Nah I don't think he goes to usf, he lives aro...	nah think go usf live

Figura 14 - Resultado do código-fonte 17, carregando dataset SPAM
Fonte: Elaborado pelo autor (2025)

Agora, vamos separar os dados de treinamento e teste, e transformar o texto em vetores utilizando o método de BoW:

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split

X = df['Message_Normalized']
y = df['Label']
X_train, X_test, y_train, y_test = train_test_split(X,y,
test_size=0.2, random_state=42, stratify=y)

# Criar o vetorizar e ajustar nos dados de treino
vectorizer = CountVectorizer()
X_train_vectors = vectorizer.fit_transform(X_train)

# Visualizar as primeiras linhas da matriz vetorizada
X_train_df = pd.DataFrame(X_train_vectors.toarray(),
columns=vectorizer.get_feature_names_out())
X_train_df.head()
```

Código-fonte 18 - Transformando dataset SPAM com BoW
Fonte: Elaborado pelo autor (2025)

Observe o resultado:

	aa	aah	aaniye	aathi	ab	abbey	abdoman	abeg	abel	aberdeen	...	zaher	ze	zealand	zed	zero	zhong	zindgi	zoe	zogtorius	zyada
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

Figura 15 - Resultado do código-fonte 18, dataset SPAM vetorizado
Fonte: Elaborado pelo autor (2025)

Por fim, podemos aplicar a técnica de aprendizado Naive Bayes com algoritmo implementado no Sklearn:

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report

# Treinar o modelo de Naive Bayes
model = MultinomialNB()
model.fit(X_train_vectors, y_train)

# Fazer previsões sobre os dados de teste
X_test_vectors = vectorizer.transform(X_test)
y_pred = model.predict(X_test_vectors)

# Avaliar o desempenho do modelo
print("Relatório de Classificação:")
print(classification_report(y_test, y_pred))
```

Código-fonte 19 - Classificador Naive-Bayes aplicado sobre o dataset SPAM vetorizado por BoW
Fonte: Elaborado pelo autor (2025)

O resultado é:

```
Relatório de Classificação:
              precision    recall  f1-score   support

         ham       0.98      0.99      0.98       966
        spam       0.91      0.90      0.90       149

    accuracy                           0.97      1115
   macro avg       0.94      0.94      0.94      1115
weighted avg       0.97      0.97      0.97      1115
```

Figura 16 - Resultado do código-fonte 19, relatório de desempenho da classificação
Fonte: Elaborado pelo autor (2025)

Na figura “Resultado do código-fonte 19, relatório de desempenho da classificação”, é possível observar que o classificador conseguiu classificar 1115 mensagens com uma acurácia de 97%. A precisão para mensagens legítimas (98%) é maior do que para identificar spams (91%), o que deve estar relacionado ao fato dos dados estarem desbalanceados (mais mensagens legítimas do que spams no dataset).

Você pode tentar trocar o modelo de vetorização das sentenças por TF-IDF e verificar se o desempenho aumenta. Além disso, é possível utilizar outros algoritmos de classificação e melhorar a etapa de pré-processamento, por exemplo, eliminando entradas vazias geradas devido à normalização.

Um contraponto interessante para os modelos de aprendizado de máquina que usam vetorização de texto são os modelos simbólicos baseados em regras e aqueles

baseados em estatística. Por exemplo, o **VADER (Valence Aware Dictionary for sEntiment Reasoning)** é um analisador de sentimentos baseado em regras e léxico, integrado à biblioteca NLTK, projetado especificamente para identificar o sentimento em textos informais como postagens em redes sociais, comentários online e mensagens curtas. Ele utiliza um dicionário pré-compilado que associa palavras a escores de polaridade (positiva, negativa ou neutra) e aplica heurísticas linguísticas para ajustá-los com base em intensificadores, negações, uso de maiúsculas, pontuação e emojis. Por ser leve, interpretável e pronto para uso, o VADER é amplamente utilizado para realizar tarefas rápidas de análise de sentimentos em língua inglesa, sem necessidade de treinamento supervisionado como nos modelos que discutimos anteriormente.

O resultado de uma Análise de Sentimentos utilizando VADER pode ser observado na figura “Resultado do código-fonte 20, análise de sentimento com NLTK VADER”. Por ser um modelo pré-treinado (mais precisamente, pré-configurado), ele tem suporte apenas para o inglês. Ele retorna uma pontuação para dimensão negativa, positiva e neutral. Esses três escores representam a proporção de palavras da frase que transmitem sentimento positivo, neutro ou negativo, ponderadas por intensidade. Assim, o VADER soma os escores positivos e negativos ajustados, e o restante da frase é classificado como neutro. Esses valores são normalizados para formar uma proporção que soma 1.

	Sentence	Negative	Neutral	Positive	Compound	Sentiment
0	I love this movie! It's fantastic!	0.000	0.264	0.736	0.8550	positive
1	This is the worst product I've ever bought.	0.369	0.631	0.000	-0.6249	negative
2	The book was okay, not great but not bad either.	0.149	0.487	0.364	0.4728	positive
3	Absolutely amazing experience!	0.000	0.313	0.687	0.6581	positive
4	I wouldn't recommend this to anyone.	0.345	0.655	0.000	-0.2755	negative

Figura 17 - Resultado do código-fonte 20, análise de sentimento com NLTK VADER
Fonte: Elaborado pelo autor (2025)

Já o **score compound** é o sentimento geral da frase, um valor único entre -1 e 1. Ele é calculado somando todos os valores de sentimento ajustados (positivos e negativos) das palavras da frase e normalizando com a fórmula: $compound = \frac{x}{\sqrt{x^2+\alpha}}$, onde x representa a soma das pontuações positivas e negativas, e α=15 é uma constante de suavização.

A seguir, temos o código-fonte para utilizar o NLTK VADER:

```
import nltk
from nltk.sentiment import SentimentIntensityAnalyzer
import pandas as pd

# Baixar o lexicon do VADER
nltk.download('vader_lexicon')

# Instanciar o analisador de sentimentos
sia = SentimentIntensityAnalyzer()

# Lista de frases para análise
sentences = [
    "I love this movie! It's fantastic!",
    "This is the worst product I've ever bought.",
    "The book was okay, not great but not bad either.",
    "Absolutely amazing experience!",
    "I wouldn't recommend this to anyone."
]

# Analisar cada frase e coletar resultados
results = []
for sentence in sentences:
    scores = sia.polarity_scores(sentence)
    results.append({
        'Sentence': sentence,
        'Negative': scores['neg'],
        'Neutral': scores['neu'],
        'Positive': scores['pos'],
        'Compound': scores['compound'],
        'Sentiment': 'positive' if scores['compound'] > 0.05
else 'negative' if scores['compound'] < -0.05
else 'neutral'
    })

# Mostrar os resultados
df = pd.DataFrame(results)
df.head()
```

Código-fonte 20 - Análise de Sentimentos com NLTK VADER
Fonte: Elaborado pelo autor (2025)

Em um comentário final, destacamos que o VADER, por ser um método baseado em léxico e regras linguísticas, realiza análise de sentimentos sem a necessidade de treinamento supervisionado. Isso o torna eficiente, interpretável e pronto para uso em textos curtos e informais, especialmente em inglês.

Em contraste, as abordagens supervisionadas usadas como Downstream Task exigem um conjunto rotulado de dados (por exemplo, sentimentos anotados para cada

frase) para treinar um modelo de classificação, como Naive Bayes ou SVM, utilizando representações vetoriais como OHE, BoW, TF-IDF ou embeddings como Word2Vec.

Enquanto OHE e BoW tratam as palavras como símbolos independentes, ignorando contexto semântico, TF-IDF pondera a relevância dos termos com base em sua frequência nos documentos. Word2Vec, por sua vez, captura relações semânticas entre palavras ao representar cada termo como um vetor denso aprendido a partir dos seus contextos. Diferentemente do VADER, essas abordagens supervisionadas tendem a se adaptar melhor a domínios específicos e a aprender nuances contextuais, embora exijam mais dados, pré-processamento e poder computacional.

4.3 Redução de dimensionalidade e agrupamento

A redução de dimensionalidade, no contexto de NLP aqui tratado, refere-se a reduzir o número de features que são usadas para representar o texto, ou seja, o tamanho do vetor que representa a sentença ou palavra. Podemos fazer uma redução de dimensionalidade por meio da análise de vocabulário, por exemplo, quando fazemos a lematização ou a estemização de um texto (análise morfológica). De maneira ainda mais simples, podemos truncar o número de palavras utilizadas para construir o vetor, escolhendo um valor arbitrário para a sua dimensão. Caso tenhamos dados rotulados, torna-se possível obter o tamanho de truncamento que otimiza a performance de um algoritmo, utilizando dados de validação para realizar testes com diferentes tamanhos de vetores.

Podemos realizar esse experimento aproveitando o exemplo de classificação de SPAM da seção anterior. Utilize os códigos-fonte “Carregando dataset SPAM e pré-processando com spaCy” e “Transformando dataset SPAM com BoW” com o código-fonte “Redução de dimensão do BoW do classificador de texto por método wrapper” para construir um gráfico de desempenho de classificação de acordo com o tamanho do vetor de BoW utilizado.

Perceba, na figura “Resultado do código-fonte 21, acurácia de classificação em função do tamanho do vocabulário do BoW”, que, acima de 100 features (palavras), o ganho de acurácia é bem pequeno, demonstrando que este pode ser um bom tamanho de vetor a ser usado. Você pode tentar outras métricas de classificação como

precisão, revocação e F1-score em cima dos SPAMs para ajustar seu modelo de classificação de acordo com esse objetivo.

```
import matplotlib.pyplot as plt
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.feature_extraction.text import CountVectorizer

# Lista de tamanhos do vetor a testar
vector_sizes = [2, 10, 25, 50, 75, 100, 125, 150, 175, 200,
225, 250, 300,400,500]

# Listas para armazenar resultados
accuracies = []

# Loop para testar cada tamanho de vetor
for size in vector_sizes:
    vectorizer = CountVectorizer(max_features=size)
    X_train_vec = vectorizer.fit_transform(X_train)
    X_test_vec = vectorizer.transform(X_test)

    model = MultinomialNB()
    model.fit(X_train_vec, y_train)
    y_pred = model.predict(X_test_vec)
    acc = accuracy_score(y_test, y_pred)
    accuracies.append(acc)

# Gerar gráfico
plt.figure(figsize=(10, 5))
plt.plot(vector_sizes, accuracies, marker='o', linestyle='-',
color='blue')
plt.title("Acurácia vs. Tamanho do Vetor (BoW)")
plt.xlabel("Tamanho do Vetor (max_features)")
plt.ylabel("Acurácia")
plt.grid(True)
plt.xticks(vector_sizes)
plt.tight_layout()
plt.show()
```

Código-fonte 21 - Redução de dimensão do BoW do classificador de texto por método wrapper
Fonte: Elaborado pelo autor (2025)

A utilização desse experimento para encontrar o melhor valor de `max_features` no `CountVectorizer` configura um método do tipo **wrapper**, uma vez que envolve a seleção de subconjuntos de características — neste caso, diferentes tamanhos do vocabulário — com base no desempenho de um modelo preditivo.

Em cada iteração, o modelo de classificação é treinado e avaliado com um número específico de características, e a métrica de desempenho (como acurácia) é utilizada para comparar as diferentes configurações. Esse processo caracteriza um método wrapper, visto que a escolha das melhores features (ou da melhor dimensionalidade) é orientada diretamente pela performance do modelo — ao contrário dos métodos *filter*, que se baseiam apenas em métricas estatísticas independentes do algoritmo de aprendizagem.

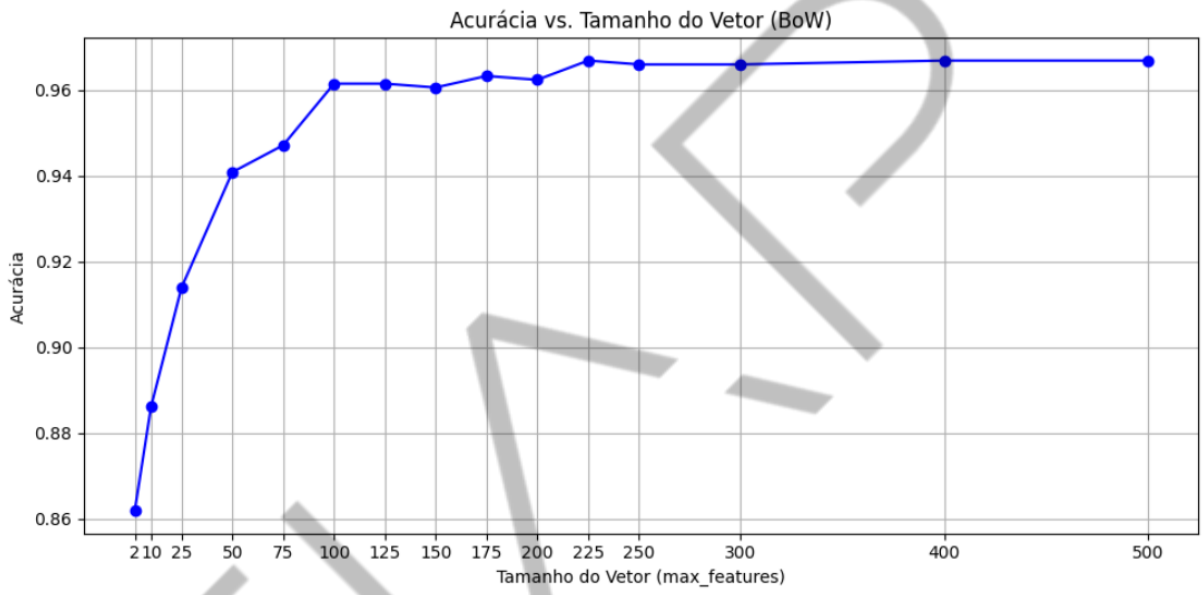


Figura 18 - Resultado do código-fonte 21, acurácia de classificação em função do tamanho do vocabulário do BoW
Fonte: Elaborado pelo autor (2025)

Também podemos reduzir a dimensão por meio de métodos não supervisionados como PCA, Truncated SVD e T-SNE. Esses métodos projetam os dados para um novo espaço dimensional, no qual os vetores de texto deixem de estar diretamente atrelados a palavras específicas do vocabulário e passam a representar configurações abstratas — como ocorre também com os embeddings do Word2Vec.

Embora o T-SNE funcione isoladamente, é comum e recomendado aplicar uma redução inicial com PCA ou SVD, a fim de melhorar seu desempenho e estabilidade. Por isso, neste exemplo — ilustrado pelo código-fonte “Redução de dimensão do BoW com método não supervisionado em cascata, Truncated SVD (LSA), seguido de T-SNE” e pela figura “Resultado do código-fonte 22, representação da redução de dimensionalidade por Truncated SVD (LSA), seguido de T-SNE” —, utilizamos uma

redução em cascata, aplicando primeiro o Truncated SVD (uma variação do PCA para dados esparsos), seguido do T-SNE para visualização em duas dimensões.

```
from sklearn.decomposition import TruncatedSVD
from sklearn.manifold import TSNE
from sklearn.feature_extraction.text import CountVectorizer
import matplotlib.pyplot as plt
import seaborn as sns

# Vetorização com 500 features
vectorizer = CountVectorizer(max_features=500)
X_vectors = vectorizer.fit_transform(X)

# Etapa 1: Reduzir de 500 → 5 com Truncated SVD (LSA)
svd = TruncatedSVD(n_components=5, random_state=42)
X_svd = svd.fit_transform(X_vectors)

# Etapa 2: Reduzir de 50 → 2 com t-SNE
tsne = TSNE(n_components=2, perplexity=40, learning_rate=200,
max_iter=1000, random_state=42)
X_tsne = tsne.fit_transform(X_svd)

# DataFrame para plotagem
tsne_df = pd.DataFrame(X_tsne, columns=['Dim1', 'Dim2'])
tsne_df['Label'] = y

plt.figure(figsize=(10, 6))
sns.scatterplot(data=tsne_df, x='Dim1', y='Dim2', hue='Label',
palette='Set1', alpha=0.5)
plt.title("Redução com LSA (5D) + t-SNE (2D)")
plt.xlabel("Dimensão 1")
plt.ylabel("Dimensão 2")
plt.grid(True)
plt.legend(title="Classe")
plt.tight_layout()
plt.show()
```

Código-fonte 22 - Redução de dimensão do BoW com método não supervisionado em cascata, Truncated SVD (LSA), seguido de T-SNE
Fonte: Elaborado pelo autor (2025)

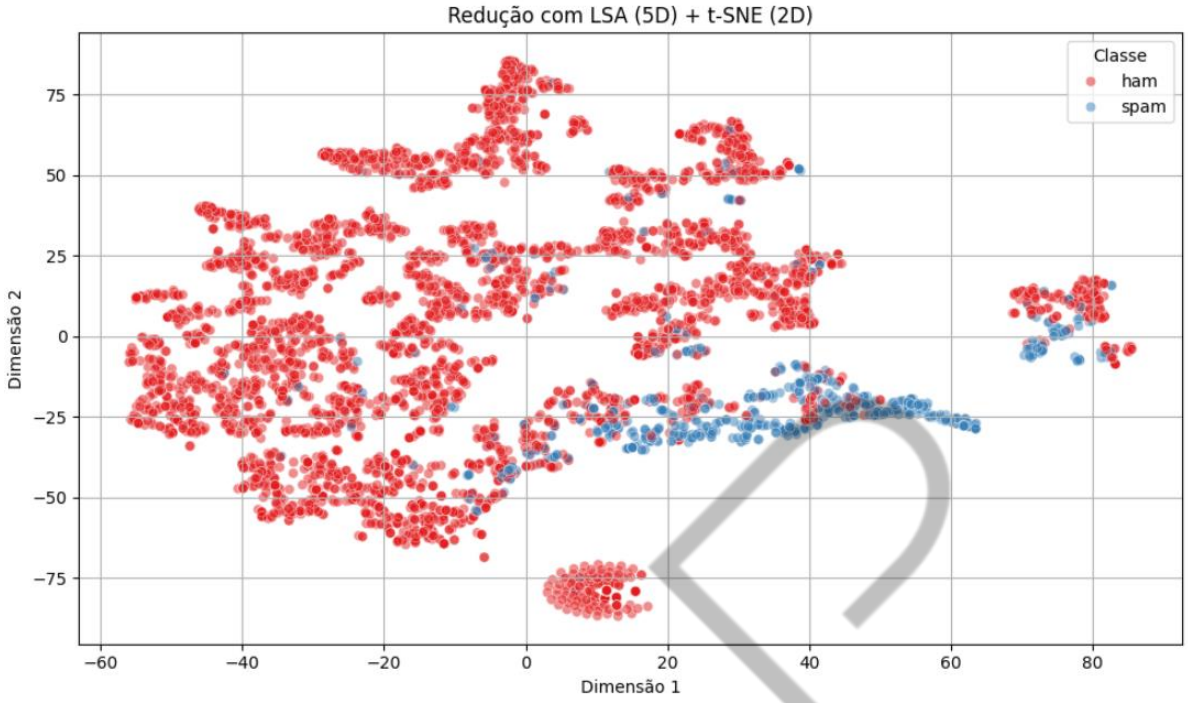


Figura 19 - Resultado do código-fonte 22, representação da redução de dimensionalidade por Truncated SVD (LSA), seguido de T-SNE
Fonte: Elaborado pelo autor (2025)

Com essa nova representação, cada frase (mensagem do dataset de SPAM) é representada por um vetor de duas dimensões. Lembre-se que fizemos a seguinte transformação: frase → vetor BoW (dimensão 500 palavras) → vetor projetado do Truncated SVD (dimensão 5, não correspondendo a palavras) → vetor projetado do T-SNE (dimensão 2, não correspondendo a palavras).

Mas por que não continuar com nossa sequência de processamento em cascata e adicionar mais uma etapa na nossa Downstream Task? Observe a figura anterior, seria possível agrupar os pontos em clusters? Isto é, encontrar grupos para cada frase dita de maneira totalmente não supervisionada. Podemos tentar! Para tanto, bastaria utilizar um algoritmo de agrupamento sobre o vetor final do T-SNE.

Vejamos o código-fonte e o resultado obtido:

```
from sklearn.cluster import DBSCAN

# Aplicar DBSCAN sobre os dados reduzidos pelo t-SNE
dbscan = DBSCAN(eps=3, min_samples=5)
clusters = dbscan.fit_predict(X_tsne)

# Criar DataFrame com resultados
dbscan_df = pd.DataFrame(X_tsne, columns=['Dim1', 'Dim2'])
dbscan_df['Cluster'] = clusters  # -1 indica outliers

# Plotar resultados do agrupamento
plt.figure(figsize=(10, 6))
sns.scatterplot(
    data=dbscan_df, x='Dim1', y='Dim2', hue='Cluster',
    palette='Set2', alpha=0.7, style='Cluster', markers=True
)
plt.title("Agrupamento com DBSCAN após T-SNE")
plt.xlabel("Dimensão 1")
plt.ylabel("Dimensão 2")
plt.grid(True)
plt.legend(title="Cluster")
plt.tight_layout()
plt.show()
```

Código-fonte 23 - Agrupamento usando DBSCAN sobre os vetores T-SNE
Fonte: Elaborado pelo autor (2025)

Observe que, na figura “Resultado do código-fonte 23, representação dos grupos encontrados pelo DBSCAN sobre o espaço projetado do T-SNE”, as cores representam grupos encontrados de maneira não supervisionada pelo DBSCAN. Isso significa que não temos certeza se essas frases possuem alguma semelhança (semântica ou sintática). Podemos selecionar algumas frases de cada cluster para tentar entender o que o algoritmo de Inteligência Artificial encontrou através dessa análise em cascata. Isto é feito no código-fonte “Mostrando as 10 primeiras frases para um grupo selecionado”.

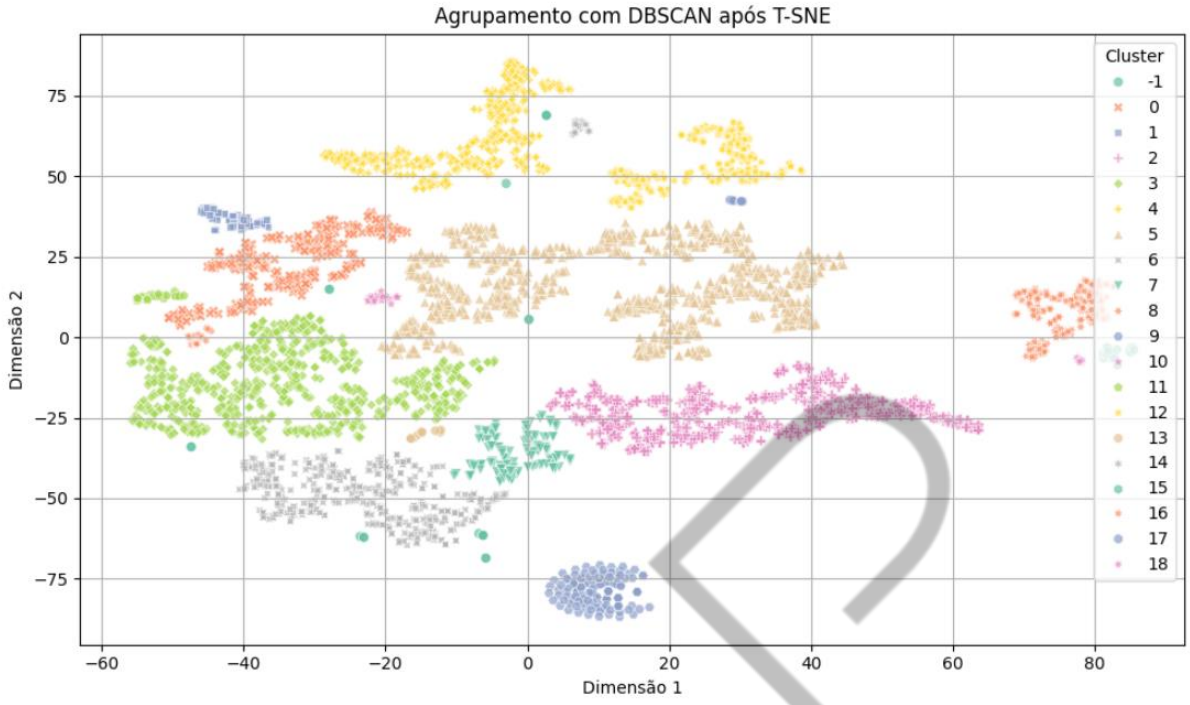


Figura 20 - Resultado do código-fonte 23, representação dos grupos encontrados pelo DBSCAN sobre o espaço projetado do T-SNE
Fonte: Elaborado pelo autor (2025)

```
df['Cluster'] = clusters
df_cluster_20 = df[clusters == 17]
df_cluster_20.head(10)
```

Código-fonte 24 - Mostrando as 10 primeiras frases para um grupo selecionado
Fonte: Elaborado pelo autor (2025)

	Label	Message	Message_Normalized	Cluster
335	spam	Valentines Day Special! Win over £1000 in our ...	valentines day special win quiz partner trip l...	17
748	spam	U are subscribed to the best Mobile Content Se...	u subscribe good mobile content service uk day...	17
779	ham	Happy New year my dear brother. I really do mi...	happy new year dear brother miss get number de...	17
1050	spam	18 days to Euro2004 kickoff! U will be kept in...	day kickoff u keep inform late news result dai...	17
2427	spam	Valentines Day Special! Win over £1000 in our ...	valentines day special win quiz partner trip l...	17
2581	spam	U are subscribed to the best Mobile Content Se...	u subscribe good mobile content service uk day...	17
2719	spam	18 days to Euro2004 kickoff! U will be kept in...	day kickoff u keep inform late news result dai...	17
3111	spam	Good Luck! Draw takes place 28th Feb 06. Good ...	good luck draw take place feb good luck remova...	17
4134	spam	FreeMsg Today's the day if you are ready! I'm ...	freemsg today day ready horny live town love s...	17
4496	ham	Hope this text meets you smiling. If not then ...	hope text meet smile let text reason smile bea...	17

Figura 21 - Resultado do código-fonte 24, apresentado as 10 primeiras frases do cluster 17
Fonte: Elaborado pelo autor (2025)

Na figura “Resultado do código-fonte 24, apresentado as 10 primeiras frases do cluster 17”, percebemos que esse agrupamento é majoritariamente composto por mensagens de spam promocional, com temas recorrentes como sorteios, assinaturas

e ofertas especiais, evidenciados pelo vocabulário comum (“valentines”, “win”, “subscribe”, “draw”). Apesar de conter algumas mensagens ham com tom pessoal e positivo, o grupo apresenta boa coerência temática, indicando que o método de agrupamento capturou efetivamente padrões semânticos relacionados a conteúdos comerciais e automático — pelo menos quando olhamos para as 10 primeiras frases do grupo 17.

O mesmo não ocorre para as 10 primeiras frases do grupo 9. Observa-se que esse grupo tem palavras erradas e poucas palavras nas frases normalizadas, indicando que precisamos melhorar nossa etapa de pré-processamento. Além disso, a falta de coerência temática dentro do grupo indica que fomos incapazes de realizar um bom agrupamento.

	Label	Message	Message_Normalized	Cluster
20	ham	Is that seriously how you spell his name?	seriously spell	9
33	ham	For fear of fainting with the of all that hous...	fear faint housework quick cuppa	9
46	ham	Didn't you get hep b immunisation in nigeria.	hep b immunisation nigeria	9
73	ham	K..k:)where are you?how did you performed?	k perform	9
74	ham	U can call me now...	u	9
78	ham	Does not operate after <#> or what	operate	9
99	ham	I see a cup of coffee animation	cup coffee animation	9
111	ham	What is the plural of the noun research?	plural noun research	9
151	ham	The wine is flowing and i'm i have nevering..	wine flow nevere	9
213	ham	K:)k:)good:)study well.		9

Figura 22 - Resultado do código-fonte 24, apresentado as 10 primeiras frases do cluster 9
Fonte: Elaborado pelo autor (2025)

Importante: processar texto com vetores de sentenças, seguido de redução de dimensionalidade e agrupamento, é desafiador devido à alta esparsidade, perda de contexto semântico e dificuldade em definir estruturas claras em espaços reduzidos. Por isso, ao usar essas técnicas clássicas, é importante estar ciente de suas limitações e complementar a análise com avaliação qualitativa ou algoritmos mais robustos para capturar relações semânticas mais sutis.

REFERÊNCIAS

SPACY. Documentação da biblioteca de processamento de linguagem natural em Python. **Spacy**, 2025. Disponível em: <https://spacy.io/>. Acesso em: 30 jun. 2025.

NLTK. Documentação da biblioteca processamento de linguagem natural em Python. **NLTK**, 19 de agosto de 2024. Disponível em: <https://www.nltk.org/>. Acesso em: 30 jun. 2025.

JONES, Karen. **A Statistical Interpretation of Term Specificity and its Application in Retrieval**, 1972.

JURAFSKY, Dan; MARTIN, James. Speech and Language Processing. 3. Ed. **Stanford**, 12 de janeiro de 2025. Disponível em:, <https://web.stanford.edu/~jurafsky/slp3/>. Acesso em: 30 jun. 2025.

MIKOLOV, Tomas; CHEN, Kai; CORRADO, Greg; DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. **ARXIV**, 07 de setembro de 2013. Disponível em: <https://arxiv.org/abs/1301.3781>. Acesso em: 30 jun. 2025.

RAJ, Sumit. **Construindo Chatbots com Python:** Usando Natural Language Processing e Machine Learning. 1. Ed. Novatec, 2019.